TÉCNICO
UNIVERSIDADE
DE LISBOA

A Process Framework for Human-AI Collaboration in the Software Development Lifecycle

Design and Instantiation of a Supporting Tool

Tomás dos Santos Taborda

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisors: Prof. Miguel Mira da Silva
Hugo de Sousa

October 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa. I acknowledge the use of artificial intelligence tools to support the drafting and refinement of this thesis, with all content produced by these tools carefully reviewed, edited, and validated to ensure accuracy and originality.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you - Thank you.

Abstract

Generative Artificial Intelligence and Large Language Models have moved from autocomplete assistants to active contributors across the Software Development Life Cycle, enabling development in which practitioners express requirements in natural language and supervise the resulting artefacts. Adoption, however, has outpaced method. A Systematic Literature Review over 646 articles confirms substantial benefits alongside recurring risks in reliability, security, traceability and accountability, and identifies the absence of structured, repeatable guidance for integrating these tools into professional workflows as the field's central unresolved problem. A majority of functionally correct AI-generated code has been reported to carry security defects, and delivery bottlenecks shift towards review rather than disappearing: value is conditional on process, not on model capability alone.

This dissertation addresses that gap through the Design Science Research Methodology. It proposes a process-oriented framework for governed Human-AI collaboration treating AI systems as first-class collaborators whose contributions must be bounded, validated and traced. The framework defines lifecycle phases with phase-specific quality gates, responsibilities expressed as functions rather than job titles, and governance metrics, anchored by a persistent versioned specification linking every generated artefact to its originating requirement. It is instantiated as Apex, a web application operationalising each phase, gate and artefact against a real project management backend. Evaluation combines application in a real project, standardised usability and workload instruments, interviews with practitioners and Agile experts, and an analytical assessment against criteria set in advance. The contribution is an evidence-grounded process model allowing organisations to adopt AI assistance without surrendering oversight of software quality, security and accountability.

Keywords

Human-AI Collaboration; Software Development Life Cycle; Large Language Models; Process Framework; AI Governance; Design Science Research

Resumo

A Inteligência Artificial Generativa e os Modelos de Linguagem de Grande Dimensão passaram de assistentes de preenchimento automático a contribuidores activos no Ciclo de Vida de Desenvolvimento de Software, permitindo exprimir requisitos em linguagem natural e supervisionar os artefactos gerados. A adopção, contudo, ultrapassou o método. Uma Revisão Sistemática da Literatura sobre 646 artigos confirma benefícios substanciais a par de riscos recorrentes de fiabilidade, segurança, rastreabilidade e responsabilização, e identifica a ausência de orientação estruturada e repetível para integrar estas ferramentas em contextos profissionais como o principal problema por resolver. A maioria do código gerado por IA funcionalmente correcto contém defeitos de segurança, e os estrangulamentos deslocam-se para a revisão em vez de desaparecerem: o valor depende do processo, não do modelo.

Esta dissertação aborda essa lacuna através da Design Science Research Methodology, propondo uma framework orientada a processo para a colaboração Humano-IA governada, cujas contribuições de IA devem ser delimitadas, validadas e rastreadas. Define fases com portões de qualidade específicos, responsabilidades expressas como funções e não como cargos, e métricas de governação, ancoradas numa especificação versionada que liga cada artefacto ao seu requisito. É instanciada no Apex, aplicação web que operacionaliza cada fase, portão e artefacto sobre uma ferramenta real de gestão de projectos. A avaliação combina aplicação num projecto real, instrumentos padronizados de usabilidade e carga cognitiva, entrevistas, e avaliação analítica face a critérios previamente definidos. O contributo é um modelo de processo fundamentado em evidência que permite adoptar assistência por IA sem abdicar do controlo sobre qualidade, segurança e responsabilização.

Palavras Chave

Colaboração Humano-IA; Ciclo de Vida de Desenvolvimento de Software; Modelos de Linguagem de Grande Dimensão; Framework de Processo; Governação de IA; Design Science Research

Contents

List of Figures

List of Tables

Acronyms

AI	Artificial Intelligence
AI-DLC	AI-Driven Development Lifecycle
BDD	Behaviour-Driven Development
CI	Continuous Integration
DRQ	Dissertation Research Question
DSRM	Design Science Research Methodology
GenAI	Generative Artificial Intelligence
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IST	Instituto Superior Tecnico
KPI	Key Performance Indicator
LLM	Large Language Model
MLOps	Machine Learning Operations
NASA-TLX	NASA Task Load Index
NIST	National Institute of Standards and Technology
OKR	Objective and Key Result
PM	Project Manager
PO	Product Owner
QA	Quality Assurance
SDLC	Software Development Life Cycle
SLR	Systematic Literature Review
SUS	System Usability Scale
UI	User Interface

1

Introduction

Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) have extended machine assistance towards generating coherent, context-aware artefacts, introducing intent-driven programming, colloquially Vibe Coding, in which developers describe desired behaviour in natural language and refine what a model produces [1, 2]. The shift is not confined to code: LLMs now participate in requirements elicitation, architectural exploration, test generation, documentation and debugging, touching every Software Development Life Cycle (SDLC) phase [3–5]. The practitioner's work moves from constructing software towards specifying intent, supervising generated artefacts and remaining accountable for them.

The evidence on outcomes, however, is more nuanced than early enthusiasm suggested, and that gap motivates this work. Adoption is close to universal while trust is not: 84% of developers report using or planning to use AI tools, up from 76%, while the proportion trusting their accuracy fell from 40% to 29% [6], with daily use around 90% [7]. That distrust is warranted, and the decisive point is that functional correctness and security do not move together: 61% of a leading coding agent's solutions were functionally correct but only 10.5% secure [8], and across more than one hundred models 45% of samples introduced an OWASP Top 10 vulnerability, with newer models improving at correctness while security stayed flat [9].

Productivity gains are equally conditional. Early evidence reported a task completed 55.8% faster with an assistant, though unpublished in a peer-reviewed venue [10], whereas a randomised controlled trial found 16 experienced developers on their own mature repositories took 19% longer while estimating afterwards that they had been 20% faster [11]; the favourable result came from a self-contained task with no codebase to respect. At organisational scale, telemetry across 1,255 teams associates high adoption with 98% more pull requests merged but 91% longer review times and no significant correlation with delivery outcomes [12]. The bottleneck moves downstream to validation rather than disappearing. Several

sources here are industry-reported rather than peer-reviewed and are identified as such throughout, so the argument rests on consistency of direction rather than any single figure: the value of AI assistance is conditional on the process surrounding it rather than a property of the model alone.

To characterise practice rigorously, an Systematic Literature Review (SLR) following Kitchenham's guidelines retrieved 646 articles, reported in Chapter 4. Its central finding is methodological: organisations lack established methodologies for the responsible use of Artificial Intelligence (AI) in professional software development [13–16], and proposed approaches remain fragmented across isolated tools, sharing no common foundation on lifecycle integration, oversight or evaluation [3]. Established methodologies do not close the gap: Waterfall, Scrum, Kanban and the Agile Manifesto were designed for environments in which every contributor is human [4], so they lack structures for governing AI participation, bounding the context a model may act upon, and allocating responsibility over generated artefacts, while their cadence assumes implementation is the binding constraint, which the evidence contradicts. **The research problem is therefore the absence of structured, repeatable guidance and standardised evaluation practices for integrating AI across the SDLC while preserving human oversight, software quality and security, and accountability.**

The objective is to design, instantiate, demonstrate and evaluate a process-oriented framework addressing that problem. Four questions structure it, labelled Dissertation Research Question (DRQ) to keep them distinct from the review questions of Chapter 4, which ask what the literature reports rather than what these artefacts achieve. **DRQ1:** how can AI-supported activities be mapped to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous? **DRQ2:** which governance mechanisms, quality gates and human-in-the-loop practices keep AI contributions traceable, validated and accountable? **DRQ3:** to what extent can those constructs be operationalised in a working system, and which resist implementation? **DRQ4:** does the framework, through that implementation, give practitioners usable and cognitively sustainable support in a real setting? Chapters 6 to 9 answer them in turn, the third including the constructs weakened, proxied or left unimplemented.

The contributions are four: the synthesis above; a process framework treating AI as a first-class collaborator, with phases, gates, function-based responsibilities, metrics and a versioned specification anchoring artefacts to requirements; Apex, a reference implementation showing those constructs implementable rather than describable; and an empirical evaluation of its usability and workload.

2

Research Methodology

Contents

This chapter presents the two research methodologies used in this dissertation, the SLR and the Design Science Research Methodology (DSRM), and maps the activities of the second onto the chapters of this document.

2.1 Systematic Literature Review (SLR)

The review follows Kitchenham's guidelines, which describe a systematic review as "a means of evaluating and interpreting all available research relevant to a particular research question, topic area, or phenomenon of interest" conducted through "a trustworthy, rigorous, and auditable methodology." The guidelines prescribe three phases: **planning**, which defines the motivation, research questions, selection criteria and protocol; **conducting**, which applies the protocol to identify and select studies; and **reporting**, which synthesises the findings.

Figure 2.1 illustrates those phases and their activities. Filtering ran over several iterations: an initial screening of retrieved articles, an abstract-reading pass to assess relevance, a filter applying Kitchenham's criteria to introductions and conclusions, and a full-text review determining the retained set. The

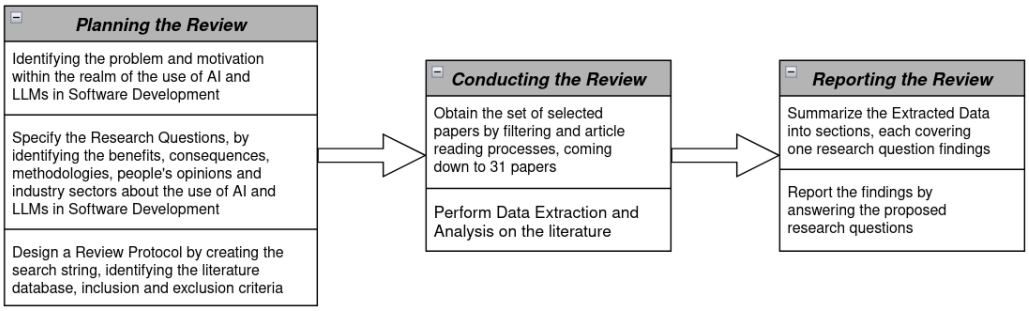

Figure 2.1: Three phases of the Systematic Literature Review

methodology was selected because it enables a structured and unbiased collection of existing research on AI-supported software development and its associated management practices.

2.2 Design Science Research Methodology (DSRM)

The DSRM concerns the development of IT artefacts, such as models, methods or frameworks, addressing identified organisational or technological problems. Unlike purely observational research it emphasises constructing solutions and evaluating their usefulness. Proposed by Peffers et al., it follows a six-step process supporting systematic design, clear contributions and transferable knowledge, which makes it suitable for research proposing process-oriented solutions. The steps are illustrated in Figure 2.2 and described below.

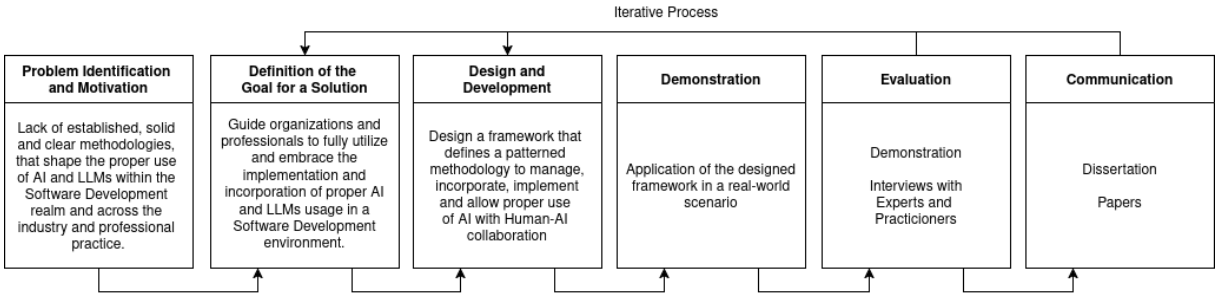

Figure 2.2: Phases of the Design Science Research Methodology

Problem identification and motivation addresses the lack of structured guidance, standardised methodologies and evaluation practices for integrating AI into software development while preserving quality, accountability and human oversight; the problem was identified through the SLR, which found fragmented practices and unclear human-AI role definitions across the SDLC. **Definition of objectives** derives from that problem objectives aimed at guiding organisations towards effective, secure and responsible adoption through appropriate processes and governance mechanisms. **Design and development** takes the review findings as input and produces the framework, defining phases, practices, hats and decision points, together with Apex as its reference implementation. **Demonstration** applies the artefact to a real project, exercising the phases end to end and producing documented artefacts, decision rationales and observations recorded against criteria fixed in advance. **Evaluation** separates

the two artefacts it assesses: the framework through interviews with practitioners and Agile experts and an analytical assessment against pre-stated criteria, the instantiation through a task-based study measuring perceived usability and cognitive workload with standardised instruments; that separation is argued in Chapter 9, since a usability result for the tool is not evidence that the method is sound. **Communication** disseminates results through a scientific article reporting the SLR, the extended abstract accompanying this dissertation, and the defence.

2.3 Mapping the Methodology onto this Document

Because DSRM prescribes activities rather than document sections, Table 2.1 states where each is carried out. The mapping is not strictly linear: as Chapter 7 describes, feedback gathered while building the reference implementation fed back into the framework's design, which is the iteration path DSRM explicitly allows.

Table 2.1: Mapping between DSRM activities and the chapters of this dissertation

DSRM Activity	Chapter	Output
Problem Identification and Motivation	Chapters 4 and 5	Evidence base from the SLR; formalised research problem
Definition of Objectives	Chapter 6	Objectives derived from the review findings
Design and Development	Chapters 6 and 7	The framework, and Apex as its reference implementation
Demonstration	Chapter 8	Application of the artefact in a real-world project
Evaluation	Chapter 9	Usability and workload measurement, interviews, analytical assessment
Communication	Chapter 10	Publications and dissertation defence

3

Research Background

Contents

This chapter presents the theoretical background of the topics associated with the research. The topics covered are Artificial Intelligence, the Software Development Life Cycle, AI in Software Development, Vibe Coding, and Spec-Driven Development.

3.1 Artificial Intelligence

Artificial Intelligence (AI) refers to systems capable of performing tasks that traditionally require human cognitive abilities, such as reasoning, decision-making, pattern recognition, or language understanding. Early AI systems followed rule-based approaches, relying on explicitly defined logic to guide decision-making. More recent developments emphasize data-driven and statistical learning, where models improve their performance by identifying patterns in large datasets. AI now underpins a wide range of applications including recommendation systems, natural language processing, predictive analytics, and autonomous systems, reflecting its expanding relevance across different industries.

Generative Artificial Intelligence (GenAI) refers to a subset of AI systems designed to create new content, rather than classify or retrieve existing information. These systems generate text, images, audio, or code based on learned patterns in their training data. GenAI models operate by predicting likely future outputs given prior context, allowing them to produce coherent and contextually relevant content. This generative capacity has positioned GenAI as a significant innovation in domains that rely on iteration, prototyping, and creative exploration, including software development.

Large Language Models (LLMs) are a specific class of GenAI systems trained on vast corpora of text and source code. Based on transformer architectures, LLMs are capable of processing and generating sequences with contextual awareness. Their primary function is next-token prediction, but this capability enables more complex tasks such as summarization, explanation, reasoning, conversational interaction, and code generation. Within software engineering, LLMs are relevant because they can interpret programming intent expressed in natural language and generate syntactically valid code in a variety of programming languages. They are also able to explain, refactor, or restructure existing code, as well as produce tests, documentation, and other boilerplate artifacts with far less manual effort. These capabilities position LLMs as collaborative tools that can support both novice and experienced developers throughout different stages of software development. [1–5, 13–39]

3.2 Software Development Life Cycle

The Software Development Life Cycle (SDLC) describes the structured phases involved in building and maintaining software systems. Therefore, the SDLC is commonly structured into six main phases: requirements analysis, where system functionality is identified through requirements engineering; design, which defines the system architecture, data structures, and component interactions; implementation, focused on writing and integrating code; testing, where correctness, reliability, and performance are verified; deployment, which delivers the system to end users; and maintenance, involving ongoing updates, improvements, and operational support throughout the system's lifetime.

The SDLC is central to this research because LLMs can engage differently across its stages. For example, LLMs can support implementation by generating code, testing by producing test cases, and maintenance by summarizing or refactoring existing code. However, the developer remains responsible for architectural decisions, validation, integration, and quality assurance.

Software development processes have evolved to emphasize iterative delivery and continuous integration, supported by methodologies such as Agile and DevOps. Despite these improvements, many development tasks remain manual, repetitive, and time-intensive. Developers spend significant time debugging, documenting code, maintaining consistency across repositories, and handling routine implementation tasks. As systems scale in complexity, these challenges become more pronounced, creating pressure to increase productivity without compromising maintainability or security. Organizations therefore seek tools that can automate or streamline specific tasks while maintaining developer oversight. This ongoing search for efficiency is a key factor driving interest in AI-assisted development workflows. [3, 4, 26, 35]

3.3 AI in Software Development

Advances in LLMs have introduced new forms of interaction between developers and software tooling. In what is often described as conversational or intent-driven programming, developers describe desired functionality in natural language and iteratively refine code generated by the model. This pattern represents a shift from writing code directly to supervising and guiding automated generation. Studies indicate that LLMs can accelerate prototyping, clarify unfamiliar codebases, and help maintain documentation quality. These capabilities can be especially valuable in early-stage development or when exploring new frameworks and libraries.

The integration of AI into the development process is increasingly characterized as collaborative. Rather than fully automating software creation, LLMs function as tools that support developers in tasks involving pattern recognition, summarization, or structural reasoning. The developer remains responsible for architectural decisions, validation, integration, and ensuring alignment with project requirements.

This collaborative model suggests a shift in the nature of developer work: from manually constructing code to curating and verifying generated artifacts. Research in this area focuses on how workflows adapt, how developers form trust in AI outputs, and how responsibilities are distributed between human judgment and automated generation. These themes are further examined in Chapter 4. [3, 4, 13, 14, 22, 26, 28, 35]

3.4 Vibe Coding

Vibe Coding refers to a style of software development in which developers describe their intentions, requirements, or constraints in natural language, and a generative model produces corresponding code or related artifacts. In this approach, the developer does not begin by writing syntactically precise code. Instead, development proceeds through a conversational interaction, where the model proposes implementations that are then reviewed and refined by the developer. This interaction model builds on the idea of human-AI collaboration introduced in the following sections. While the model generates initial drafts or prototypes, the developer evaluates correctness, aligns the output with project requirements, and iteratively modifies the prompt or the produced code.

Vibe Coding shifts part of the programming effort from manual construction to specifying intent and assessing alternatives, which can influence how tasks are conceptualized and distributed during development. The approach does not replace traditional programming but reframes how initial structure and exploratory code are produced. As such, it has implications for developer workflows, cognitive load, and the distribution of effort across the SDLC. These implications, including reported advantages and limitations, are examined in detail in the Systematic Literature Review reported in Chapter 4. [1, 2]

3.5 Spec-Driven Development

Spec-driven development is an emerging engineering practice in which a persistent, versioned specification, rather than the source code itself, is treated as the primary artefact that both developers and AI agents work against. Instead of prompting a model directly against a codebase and accepting whatever

it produces, the practice interposes a written specification, of requirements, design decisions, and constraints, that the model must consult and satisfy before code is generated, and that is kept in sync with the codebase as it evolves. Industry examples of this pattern include AWS Kiro's three-stage requirements, design, and tasks workflow, GitHub's Spec-Kit, which adds a "constitution" of immutable project principles that generated code may not violate, and the Tessl Framework, in which, as its own materials put it, the specification itself becomes the maintained artefact rather than the code [40].

The practice is tracked by Thoughtworks' Technology Radar as an emerging AI-assisted engineering technique, named specifically "spec-driven development" [40], and has begun to receive academic attention: Farrag frames it as specification-driven governance, arguing that it addresses what is termed the productivity-reliability paradox in AI-augmented development, though this framing is currently reported only in a preprint that has not yet completed peer review [41]. The practice responds directly to the risk profile characterised in Chapter 1: narrowing and anchoring the context a model is permitted to act upon is a governance mechanism, not merely a convenience, since it gives generated output a stable reference to be validated against rather than leaving correctness to be inferred from the prompt alone. This precedent is the direct grounding for Spec-Anchored Continuity, the persistent specification mechanism at the centre of the framework proposed in Chapter 6.

4

Systematic Literature Review

Contents

In this section, the execution process of the systematic literature review is described. The description follows the SLR stages, which are as follows: Planning, Conducting, and Reporting the results.

4.1 Planning the Review

This section details the planning stage of the SLR methodology. The motivation for the review is presented, then the research questions addressed in the search, and lastly the search method employed.

4.1.1 Motivation

The integration of AI, particularly LLMs and GenAI, represents a significant transformation in Software Engineering. These technologies are increasingly adopted across the SDLC to support activities such as code generation and completion, documentation, requirements engineering, automated bug fixing, and developer learning. Their use has been associated with notable productivity gains, improved efficiency, and enhanced support for complex development tasks by augmenting traditional development tools and information retrieval practices [3, 4, 13, 14, 22, 26, 28, 35].

Despite these benefits, the rapid adoption of AI and LLMs in software development has introduced unresolved challenges that motivate systematic investigation. The literature highlights persistent concerns related to the quality, reliability, and trustworthiness of AI-generated artifacts, as well as technical and methodological barriers such as limited explainability and the black-box nature of advanced models. In addition, effective utilization of LLMs remains highly dependent on prompt formulation, while existing research often focuses on isolated tasks, leaving several SDLC phase,-particularly requirements engineering and design, with limited practical guidance.

Therefore, the motivation for this Systematic Literature Review is to comprehensively map and synthesize existing research on the use of AI and LLMs in Software Development. By organizing current evidence on benefits, limitations, methodologies, stakeholder perspectives, and application domains, this review aims to identify research gaps and inform both researchers and practitioners on how to better leverage AI technologies across the SDLC.

4.1.2 Research Questions

At the start of the Planning the Review section of this SLR, there were a lot of research questions that could have been written about this overly general topic of AI adoption in the development process of applications. After making a throughput selection of the Research Questions that were more relevant and specific about the theme the Chosen RQs pool became a lot more concise for this Systematic Literature Review. As such, this systematic review plans to answer the following RQs:

Table 4.1: Research Questions and Focus Areas

RQ	Research Question	Focus
RQ1	What are the benefits of using and incorporating Vibe Coding, AI, and Large Language Models in software development?	Identify and categorize improvements in productivity, creativity, code quality, automation, collaboration, and innovation enabled by AI-driven tools.
RQ2	What are the consequences that may arise from using and incorporating Vibe Coding, AI, and LLMs in software development, both in code generation and other sections?	Analyze risks such as technical debt, bias, reduced developer learning, code ownership, ethical issues, and dependency on AI tools.
RQ3	What methodologies and management methods are being used to integrate Vibe Coding, AI, and LLMs into software engineering processes and workflows?	Explore frameworks, best practices, and process models (e.g., AI-augmented Agile, human-in-the-loop development, prompt engineering practices).
RQ4	What are developers' and other stakeholders' opinions regarding the use of AI and LLMs in software development?	Assess perceptions, trust, satisfaction, and acceptance of AI-assisted coding among developers and organizations.
RQ5	Which industry sectors are adopting AI for application development, and how do they benefit from its integration?	Map adoption trends across industries (e.g., finance, healthcare, IT, education), highlighting sector-specific motivations and outcomes.

4.1.3 Review Protocol

The systematic literature review starts with a literature search. A search string is defined to then be used in the data source of choice. The search criteria used for this review resulted in 646 articles to review.

Table 4.2: Research Elements and Details

Element	Research Details
Source	EBSCO
Final Search String	(“Vibe Cod*” OR “AI models” OR “AI agen*” OR “large language model*” OR “generative AI” OR openai OR “chatgpt” OR LLM* OR “claude” OR “gemini” OR copilot OR co-pilot) AND (“software development” OR “application development” OR “code generation”)
Search Method	Search string found in the abstract of articles in English academic journals or conference materials, with no date range limit
Results	646

The aim of this review was to research the use of AI/LLMs in developing applications; but that specific topic can be written in many different ways. When talking about this theme, a very recent term used in the world of Software Development came to mind: "Vibe Coding". For such a new topic and specific term regarding the use and adoption of AI in the development process of software, there were used a lot of keywords in the search string that have the same significance and could replace it since the topic was recent but its methodology wasn't, it was still used but with different expressions and the term "Vibe Coding" itself didn't provide a lot of relevant articles covering the theme. Keywords such as "code generation" and "application development" used in conjunction with "generative AI" and "AI models" provided a search string that could get the desired world of use and adoption of AI and LLMs in the software development process as a whole.

The defined search string was then used to query the EBSCO Online Digital Library. For this review, the “Advanced Search” option was used and the query was executed in the AB abstract field and title of the papers. To limit the number of articles found, the search was limited to articles in academic journals or conference materials that had the full text available and were in English.

4.2 Conducting the Review

This section covers the second stage of the SLR. To conduct the review, the 646 articles extracted in the previous stage were analyzed according to the criteria used for the filtering process. The criteria used are detailed and a representation of the results found along the process is provided.

4.2.1 Filtering Process

The filtering process starts with the 646 articles that resulted from the search string query and Figure 4.1 shows that exact process. Extracting and downloading those 646 articles was impossible due

to EBSCO's web page loading capacity which only allowed the download of 50 articles at a time; after reloading and extending the page to its maximum, the website could only download and display, by relevancy order, 467 out of the 646 articles obtained from the search string.

To organize the articles and to help the filtering process, the AI-Powered Systematic Review Management Platform "Rayyan" was used. From the 467 extracted, 227 duplicates were identified and removed, resulting in 350 articles left for Abstract Reading. The abstract of the resulting articles was read to ascertain the topic covered and 286 articles were rejected due to the reason of covering another topic other the overall theme of "Developing applications using AI" leaving exactly 64 articles.

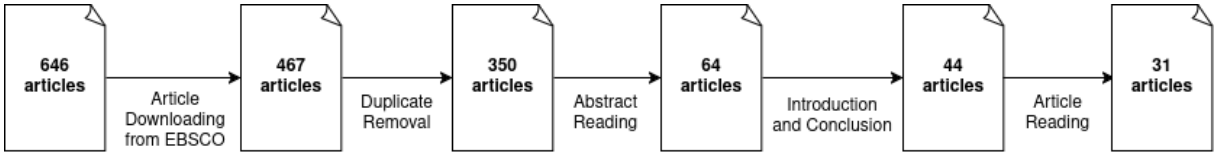

Figure 4.1: Articles Filtering Process

After making sure that all the relevant papers concerning the theme were still being considered, it was time to properly read the content of these papers, and during this section these 64 articles were separated into 4 categories: Included, Maybe, Excluded and Skipped.

The "Skipped" articles, although they passed through the abstract reading phase, after reading the introduction and conclusion of those papers and evaluating them on regards if they are relevant to the theme, if they are relatively recent (which is important due to the highly dynamic and fast-evolving nature of the field) and if any of the five research questions were answered, there were 20 skipped papers and properly excluded. The number of "Maybe" articles was always changing during the article reading section, but the files that were classified as "Maybe" was due to not knowing if its content would be relevant to the theme in question. Most of them were properly excluded due to prioritizing more recent studies to reflect the current state of the art and the papers that fit into the topic were included.

If the reviewed sections of an article provided a clear and explicit answer to any of the five research questions, the article was included in the study. However, when the information presented was unclear, insufficient, or lacked the necessary detail to confirm whether an answer was provided, the predefined inclusion and exclusion criteria were applied to determine eligibility. These criteria are summarized in Table 4.3. From the initial set of 64 articles, after excluding and deciding which "Maybe" articles were relevant, 31 met the criteria and were retained for analysis.

Table 4.3: Inclusion and Exclusion Criteria

Inclusion Criteria	Exclusion Criteria
• Relevance regarding AI and LLM usage in the development of software applications • Relevance regarding benefits and consequences of incorporating and using AI and LLMs in the development of software applications • Relevance regarding methodologies and management methods used to integrate AI and LLMs into software engineering processes and workflows • Relevance regarding people's opinions with AI and LLMs use in software development • Relevance regarding which industry sectors are adopting AI for application development and its benefits	• Not written in English • Does not clarify any of the Inclusion Criteria • Outdated due to the highly dynamic nature of the field (anything older than 2024)

4.2.2 Data Extraction Analysis

Having selected the articles, these can now be analyzed, subdivided according to the venue in which they were published, journal or conference material, as well as identifying the conferences and journals used and the year it was published. Curiously enough, none of the selected papers were categorized as Conference Materials, only Peer Reviewed Academic Journals categorized as such by EBSCO were selected and extracted as shown in Figure 4.2.

4.3 Reporting the Review

This section covers the third and last phase of the SLR methodology, reporting what the analysis of the retained articles established and answering each research question in turn. The distribution of the retained corpus by publication venue is given in Appendix A.1; it is concentrated rather than diffuse, with ACM Transactions on Software Engineering and Methodology contributing ten of the retained articles and Information and Software Technology three, while the remaining venues contribute one each, which is consistent with a topic that is being actively published across the field rather than owned by any single outlet.

4.3.1 Benefits of AI and LLMs usage in Software Development

The first research question looks into the benefits of using and incorporating AI and Large Language Models in software development to developers, companies and other stakeholders. The benefits found

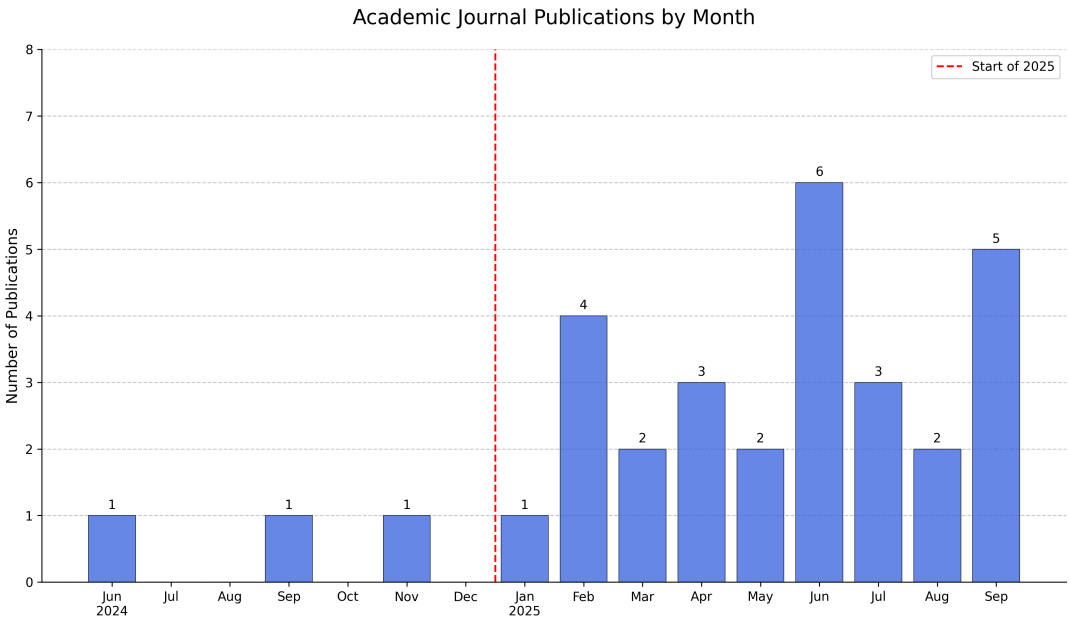

Figure 4.2: Academic Journal Publications

in the literature have been collected in Table 4.4. To answer this question, this section will analyze the benefits found.

Table 4.4: Benefits of using AI and LLMs in Software Development

Benefits	Description	#	References
Improved Code Quality, Accuracy, and Traceability	"In practice, it has been proven that using Generative AI can increase efficiency and productivity by improving code quality and optimizing work with faster times." [14].	20	[3, 4, 13–16, 18, 19, 21–24, 26, 27, 29–31, 33, 34, 36]
Increased Productivity and Efficiency	"Accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles" [1].	18	[1–4, 13, 14, 19, 20, 24, 27, 29, 30, 33–37, 39]
Automation of Testing, Debugging, and Repair	AI and LLMs "can increase the efficiency of software development by automating coding, bug detection, debugging, and personalizing software according to user needs" [14].	12	[1, 3, 16, 17, 20,21,29,30, 33–35, 37]
Optimization of Code Performance and Efficiency	"The proportionally small number of studies in SE subfields like Build, Release, Deploy, Operate and Monitor indicates a broad untapped area for cost savings and novelty, which could bring major optimisation in Software Engineering" [28].	10	[1, 2, 4, 13, 14,17,24,28, 33, 35]

Continued on next page

Benefits	Description	#	References
Support for Early Software Development Phases (Requirements & Design)	AI and LLMs have effectiveness in "...enhancing stakeholder communication and requirement documentation" while noting challenges with bias and accuracy in requirements and design elicitation [29].	8	[4,15,16,22, 27,29,33,35]
Automated Documentation and Summarization	"LLMs can [...] automatically generate documentation for codebases, including explaining the purpose and functionality of specific functions, classes, and modules. This ensures that documentation stays current and can be updated as code evolves, providing developers with easily understandable, well-structured explanations of how various parts of the system work." [24].	8	[4,13,17,22, 24,26,35,36]
Enhanced Security and Vulnerability Management	Gen AI technology good practices enhances SDLC security by supporting development through "vulnerability detection, threat modeling, secure coding practices, and incident response" [17].	6	[17, 18, 26, 33,35,39]
Knowledge Acquisition and Understanding	"Reduce technical barriers and transform the programming process into a collaborative, conversational, and more accessible act" [2].	5	[2,13,20,27, 33]
Management and Organizational Support	Assist in managing and "curating organizational information about day-to-day organizational practices" [27].	5	[3, 4, 16, 17, 22]
Developer Well-being and Satisfaction	AI tools improve developer satisfaction by enabling developers to "prioritize interesting and fun tasks through automating menial, boring tasks" [27].	4	[4,19,27,33]
Democratization and Accessibility	AI integration aims to "shift the programming paradigm from a traditional code-centric approach to one that involves verbal interactions between the programmer and the artificial intelligence agent being used", making programming more accessible [2].	2	[1,2]

The integration of AI and LLMs into software development yields profound benefits, foremost among which is the dual improvement of productivity and code quality, which appear to be mutually reinforcing and more abundant within the articles. This acceleration does not come at the cost of quality; on the contrary, the literature indicates that AI helps in generating code that adheres to high standards, which directly increases the quality, accuracy, and particularly the traceability of the final product. [30]. This synergy is further supported by the automation of traditionally time-consuming quality assurance tasks like testing, debugging, and repair, where AI's ability to refine implementations and correct errors significantly reduces manual intervention and boosts the code's overall robustness; specifically, LLMs leverage explicit feedback, such as compiler error messages, in follow-up prompts to iteratively fix their own coding errors within the same conversation context.

Beyond the direct impact on the codebase, the integration of AI yields substantial benefits for the developers themselves by improving job satisfaction and democratizing programming knowledge. Although

being the benefit that is mentioned less in the reports, its equally important to the art. By automating monotonous and menial tasks, AI tools allow developers to prioritize more engaging and intellectually stimulating work that requires professional judgment, which in turn enhances their well-being and satisfaction. Concurrently, AI acts as a catalyst for knowledge acquisition and collaboration. It lowers technical barriers and helps transform the programming process into a more conversational and accessible activity, making it easier for developers to understand complex codebases. This evolution fosters a more inclusive environment, shifting programming from a purely code-centric discipline to an interactive one between the programmer and an intelligent agent, thus making it more accessible to a broader audience.

The advantages of incorporating AI and LLMs are not confined to a single phase but extend across the entire SDLC. In the crucial early stages (Requirements Analysis and Design Modeling), these technologies enhance stakeholder communication and aid in the elicitation and documentation of requirements and design specifications, for example, by transforming user stories into formal models like UML diagrams [15]. The benefits continue into the implementation and maintenance phases, with AI tools specifically designed to optimize the computational performance of generated code, improving its runtime and memory usage, often exceeding typical human performance. Furthermore, AI provides critical support for non-functional requirements, most notably through enhanced security and vulnerability management via support for threat modeling and secure coding practices. Finally, the persistent challenge of documentation is addressed by LLMs that can act as summarizers to automatically generate multi-intent comments and other documentation artifacts, ensuring the long-term maintainability of the software.

4.3.2 Consequences of AI and LLMs usage in Software Development

This second section will analyze, cover and show more about the consequences that may arise while using and incorporating AI and Large Language Models in Software Development both in code generation and other sections of the art. The consequences found in the many articles have been collected in Table 4.5.

Table 4.5: Consequences of using AI and LLMs in Software Development

Consequences	Description	#	References
Increased Operational Costs and Resource Constraints	"The financial burden can become substantial and must be considered when LLMs are deployed to handle a large number of tasks" [5].	20	[3, 5, 20, 22–24, 26, 28, 35, 38]
Reliability, Accuracy, and Hallucination Risks	This is particularly problematic in complex or niche scenarios. "Because this AI can simply make things up or sometimes spit out things that don't even exist" [13].	15	[2, 4, 15, 16, 18–20, 22, 23, 30, 31, 34–37]

Continued on next page

Consequences	Description	#	References
Security Vulnerabilities in Generated Code	"Recent studies suggest that code generated by AI models often contains security vulnerabilities that may follow patterns different from those in human-written code. Evidence suggests that AI-generated code may harbor unique security risks in areas such as cryptographic implementation, memory management, and error handling" [39].	15	[3,13,14,16–20,24,26,28,33,36,37,39]
Ethical, Legal, and Intellectual Property Issues	"Questions about the ownership and licensing of that code will arise, particularly when the models are trained on publicly available, open-source projects" [24].	8	[2, 3, 13, 17, 19,22,24,35]
Homogeneity and Bias Propagation	As LLMs increasingly train on LLM-generated code, there is a risk of code homogeneity, reducing diversity in coding practices and potentially perpetuating slow, biased, or insecure patterns. This reduces the diversity of code practices over time, as LLMs tend to "reaffirm patterns seen in their training data (e.g., public GitHub repositories)" [35].	8	[3,13,14,17, 19,24,34,35]
Evaluation and Standardization Challenges	"It is urgent to establish an effective, objective, and complete evaluation system for large language models for code" [3].	7	[3,14,15,23, 26, 32, 39]
Risk of Over-reliance and Cognitive Decline	"Of course, sometimes sometimes there is almost the danger that you stop thinking for yourself a little. I think that might also be a problem in the future, that if AI is constantly available, people will tend to become lazy and stop thinking for themselves" [13].	4	[2,13,19,27]
New Overhead and Workflow Inefficiencies (Human Review)	Integrating AI requires developers to shift their focus, often spending more time reviewing, verifying, and refining outputs, rather than writing new code because LLMs still "require human oversight and review of their outputs before implementation" [29].	4	[2,13,22,29]
Ineffectiveness of Existing Security Tools	Existing tools detect only "46.7% of vulnerabilities in AI-generated code, a significantly lower rate than for human-written code vulnerabilities (66.4%)" [39].	3	[17, 26, 39]

The integration of GenAI and LLMs into software development introduces significant technical, ethical, and operational consequences that demand careful mitigation and human oversight with operational and technical risks being the most frequently cited in the literature, as detailed in Table 4.5. Operational efficiency is threatened by increased costs and resource constraints. The computational and financial costs for training, fine-tuning, and deploying these large-scale models can be substantially higher than traditional methods, potentially offsetting the anticipated productivity gains.

Another major technical concern revolves around the inherent limitations of these models concerning reliability, accuracy, and security which was the second most referenced consequence in the literature. LLMs are fallible and frequently produce erroneous outputs, commonly referred to as "hallucinations", which involve the generation of plausible but incorrect or fabricated information. This is particularly

problematic when tackling complex, niche or specialized programming scenarios. Furthermore, current security analysis tools struggle to detect these unique AI-generated vulnerabilities effectively, only identifying about 46.7% of issues in AI code compared to 66.4% in human-written code, suggesting a fundamental mismatch between existing frameworks and the new types of flaws introduced by AI [39]. The resultant volume of potentially low-quality or insecure AI-generated code necessitates implementing scalable and specialized testing and quality assurance processes to maintain the integrity of software systems. Although being mentioned less in the articles, this consequence is equally important as the other consequences mentioned in the literature and ties directly with the the prevalence of the Reliability, Accuracy, and Hallucination Risks with AI and LLMs .

Beyond the technical domain, the rapid adoption of AI introduces profound risks related to human factors, ethics, and economic viability. A primary worry is the heightened risk of over-reliance on AI, which may lead to a reduction in developers' critical thinking skills, independence, and overall software engineering expertise. Developers, accustomed to instant assistance, may become reluctant to engage in complex coding or troubleshooting, resulting in a gradual loss of core development skills. Ethical challenges include the potential for biases embedded within training data to be perpetuated and magnified in generated code, particularly critical in sensitive industries [24]. Furthermore, legal and intellectual property issues pose major obstacles, as it remains unclear who owns the code generated by AI models and how to ensure compliance with licensing requirements, especially when models utilize open-source code for training [35].

These challenges highlight the essential and continuing role of human developers, especially in performing rigorous verification and maintaining quality throughout the SDLC. The inherent non-deterministic nature of LLMs, where the same prompt can yield different outputs, coupled with high output variability, mandates extensive human oversight and review before integrating AI-generated artifacts into production. Developers must account for the time spent correcting inaccuracies or mitigating unexpected vulnerabilities, which can counteract the intended efficiency gains. The process of prompting and refining AI outputs may require more time than anticipated. Moreover, the lack of standardized benchmarks and evaluation metrics makes it challenging to objectively assess the genuine contribution of AI tools to quality assurance and security within the SDLC, hindering accurate comparison and validation of their effectiveness. Therefore, successful integration requires not only highly effective prompting strategies but also specialized tools and methodologies designed to manage the complexity and risks associated with auto-generated code and support within a SDLC professional environment.

4.3.3 Methodologies and management methods used to integrate AI and LLMs in Software Development

This section will aim to answer to the third research question and analyze the literature in order to list all the methodologies and management methods used to integrate AI and LLMs in Software Development.

Table 4.6: Methodologies and Management Methods used to integrate AI and LLMs in Software Development

Methodologies	Description	#	References
In-Context Learning (ICL) / Few-shot Learning	"ICL seeks to improve the abilities of LLMs by integrating context-specific information, in the form of an input prompt or instruction template, during the inference and thus without the need to perform gradient-based training" [38].	9	[15, 16, 20, 22, 26, 33–35, 38]
Prompt Engineering (PE) / Prompt-Driven Development	"The effectiveness of prompt engineering critically determines the performance of leveraging the capabilities of LLMs". [26]	5	[3,20,26,32, 33]
Chain-of-Thought (CoT) Prompting	"A CoT is several intermediate natural language reasoning steps that lead to the final answer... the CoT only uses natural language to describe how to solve a problem step by step sequentially". [31]	5	[3,16,20,26, 31]
Human-In-The-Loop (HITL) Paradigm	"While overall, LLMs exhibit decent reliability in their outputs with varying degrees, they still require human oversight and review of their outputs before implementation". "The importance of the human contribution in identifying and resolving problems to maintain software quality is widely acknowledged by many authors, which also gives rise to the Human-in-the-Loop (HITL) paradigm" [29].	5	[15, 24, 29, 35, 37]
Structured Prompting / Structured Chain-of-Thought (SCoT)	"SCoT denotes several intermediate reasoning steps constructed by programming structures", and "SCoT prompting asks LLMs first to generate an SCoT and then output the final code." [31]	4	[23, 26, 30, 31]
Multi-Agent Systems / Self-Collaboration Approach	"Different agents play different roles and collaborate to fulfill user requirements". "[...] a self-collaboration framework with role instruction, which allows LLM agents to collaborate with each other to generate code for complex requirements" [21, 37].	4	[13, 18, 21, 37]
Integration with External Tools and Domain Knowledge	"Combining the linguistic capability of LLMs with specialized domain knowledge to enhance output quality, functionality, or security". "The integration of relevant context into prompts may further improve current GenAI models, thereby reducing hallucinations and increasing reliability". [13, 37]	3	[3, 4, 13, 37]
AI-Assisted Pair Programming / Human Oversight	"LLMs still require human oversight and review of their outputs before implementations, and they present unique characteristics to be considered before adoption". "The AI orchestrator will coordinate the perception and action of each agent" [29, 35].	3	[2, 29, 35]
Retrieval-Augmented Generation (RAG)	"RAG have gained attention. ICL and RAG offer a low-computation option by eliminating the need for parameter updates". "RAG demonstrates higher effectiveness than ICL but falls short of LoRA's effectiveness across all three CodeLlama model variants". [38]	2	[20, 38]

Continued on next page

Methodologies	Description	#	References
AI-Augmented Agile Development (e.g., AgileGen Framework)	"[...] we propose a human-agent collaborative generative software development approach, focusing on human involvement at the beginning (scenario decision-making) and end (acceptance and recommendation decision-making) of each iteration" [4].	1	[4]
Utilizing Gherkin Language (Behavior-Driven Testing)	"[...] we use Gherkin to supplement unclear user requirements with well-defined acceptance criteria, guiding the generation of applications that align with user requirements" [4].	1	[4]
Vibe Coding	"[...] emerging concept of vibe coding, in which artificial intelligence (AI) accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules" [1]	1	[1]
Integration of LLMs with Traditional Testing	"the integration of LLMs with traditional approaches has emerged as an important direction for future exploration". "By combining them together, there can be 1.4× to 2.3× improvement than the baselines". [37]	1	[37]

The integration of LLMs and GenAI into software engineering has necessitated the creation of structured methodologies and management paradigms focused on optimizing model performance, defining human-AI collaboration, and formalizing the Software Development Lifecycle itself. This analysis section details the core integration methods, ranging from subtle parameter tuning to large-scale, process-oriented frameworks.

The fundamental layer of integration, and a clear focus of the literature, relies on methods to improve model interaction by prompts. This is led by In-Context Learning (ICL), which, with 9 articles, is the most frequently discussed technique, seeking to improve LLM abilities by integrating context-specific information during inference.

Closely related is Prompt Engineering (PE), recognized as the practice of designing inputs to elicit reasoning and knowledge from LLMs for specific tasks. The effectiveness of LLMs is critically determined by the quality of these prompts. This principle led to the adoption of advanced reasoning techniques like Chain-of-Thought (CoT) prompting, where the model generates intermediate natural language steps leading to the final answer and building significantly upon CoT is Structured Chain-of-Thought (SCoT), another prompting technique that explicitly introduces programming structures (such as sequential, branch, and loop structures) into the reasoning steps. [31]

Given the inherent limitations of LLMs, including their overall output reliability and high variability, the Human-In-The-Loop (HITL) paradigm is considered mandatory for maintaining software quality [29]. HITL ensures human developers maintain critical oversight by reviewing and verifying AI-generated outputs before they are implemented, recognizing the continued necessity of human contribution in resolving problems. This human-AI teamwork approach is formalized in frameworks such as AI-Augmented Agile Development (AgileGen) which is mentioned once [4], which structures collaboration into lightweight iterations and reserves key human decision-making at the beginning (scenario

definition) and end (acceptance and recommendation) of each cycle. This management strategy prevents the accumulation and propagation of errors that can occur in fully automated multi-agent systems. The one thing that is prevalent and that should always be a non-negotiable, is the presence of a developer that is always verifying AI-generated outputs before they are implemented to always ensure good and secure quality code and development throughout the whole Software Development Lifecycle.

Finally, "Vibe Coding" which is mentioned only once in the literature, and ties directly with the use and adoption of AI in the development process of software. This paradigm represents a conceptual leap in prompt application, accelerating the development process, particularly for domain-specific tasks like biomedical software and other sectors in the industry, by translating abstract intent and natural language directly into functional code and applications. Although Vibe Coding is relatively recent and is not mentioned a lot in the literature, its becoming the new "trend" and its the evolution of prompt based development, however its important to say that "Vibe Coding" has issues and a lot of problems, translating these "vibes" reliably requires strict use of methodologies and methods like the ones that were collected in this review to ensure better results [1].

4.3.4 Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development

The fourth research question concerns the human perspective, and the reception of these tools proved to differ enough between groups that the extracted opinions are tabulated separately by group, in Appendix A.2: the public and researchers, developers and practitioners, and organisations and management. The separation is itself a finding, because the three groups do not agree.

Among the public and researchers, the dominant position is scepticism grounded in fallibility rather than in capability. General perception, cited in six articles, treats these systems as a mixture of software and sorcery, and the primary technical barrier reported is misinformation risk: generated code may execute without error while failing to meet the intended requirement. That risk is amplified by variability, since the same prompt repeated yields different code each time. This is what makes the single most prevalent opinion across all three groups, the need for human oversight with seven articles, follow from the evidence rather than from caution: errors such as inconsistencies in design diagrams or incomplete implementations propagate into project integrity.

Developers and practitioners report the opposite emphasis. Productivity and efficiency gains are the most-cited positive opinion for this group, in six articles, and LLMs are valued for knowledge acquisition because they allow a developer to bypass extended search or dependence on colleagues [13]. Two countervailing findings appear alongside it. The convenience carries a documented risk of over-reliance and erosion of technical competence, and effectiveness is not seamless: prompt design is reported as a significant time investment requiring considerable skill, which reduces the efficiency it promises [13].

Organisations and management treat adoption as a risk-management exercise. The strategic case is tied to job satisfaction, talent retention and competitiveness, since automating routine work retains valuable staff [27], but it is constrained by the group's most-cited concern, data security and privacy, in six articles. The consensus strategy is to avoid commercial LLMs in favour of open models fine-tuned on organisation-specific data, which in turn raises operational cost and pushes teams towards medium-sized models to manage computational limits. Expectation management appears less often but matters

for the same reason: organisations report needing to discount the hype surrounding these tools when planning their integration.

Read together, the three groups converge despite their different emphases. Enthusiasm about productivity is never reported without an accompanying concern about reliability, and every group independently arrives at human oversight as the necessary control. That convergence, from stakeholders with different incentives, is what makes oversight a requirement to be designed for rather than a preference to be accommodated.

4.3.5 Industry sectors adopting AI and LLMs for Software Applications Development

The fifth research question asks which industry sectors are actively adopting AI for application development and what sector-level benefits they report. The extracted evidence is tabulated in Appendix A.3; what follows is what it establishes.

Adoption concentrates overwhelmingly in Software Engineering, Software Development and IT Consulting, which is unsurprising since these domains are both the origin and the primary proving ground for LLM-based tools. The reported use cases centre on code generation, completion, explanation and refactoring, and several sources describe a gradual shift in the developer role from writing code manually to orchestrating AI-enabled workflows in which the human directs, validates and refines automated output [4, 27, 29]. In consulting specifically, LLMs are valued for accelerating onboarding and transferring client-specific business knowledge between projects. Adoption also appears in specialised technical domains involving infrastructure-level or mathematically intensive systems, where models must be adapted or augmented with structured knowledge to produce reliable output [3, 5, 17, 37].

Beyond the core **IT!** (**IT!**) sector, adoption clusters in high-stakes regulated environments: Finance, Banking and Regulated Industries in eight articles, Transportation and other safety-critical domains in seven, Healthcare and Biomedical Research in five, and Legal Reasoning in four. In these domains LLMs are introduced cautiously, with emphasis on reliability, auditability and regulatory compliance, and the literature is consistent that extensive human review and verification remain mandatory wherever incorrect or unverifiable output could produce safety risks, compliance failures or legal liability [1, 3, 26, 33]. Education and Training, in five articles, is an emerging area in which LLMs act as adaptive learning companions rather than replacements for educators, supporting exercise generation, personalised feedback and formative assessment [2, 15, 34].

The pattern across sectors is the same one the earlier questions produced from a different direction. Where the cost of an incorrect output is low, adoption is broad and the reported benefit is speed; where that cost is high, adoption is conditioned on verification. No sector reports adopting these tools without a human check, and the sectors with the most to lose report the most explicit checks. That is a finding about process rather than about capability, and it is the point at which this review begins to constrain the design of a framework rather than merely describing practice.

4.4 Discussion

This section discusses the relevant information extracted from the articles selected for the SLR, synthesizing the findings from the five research questions to construct an integrated and broad view of the current landscape of AI and LLM adoption in software development. The results confirm the innovative and transformative potential of these technologies, but also highlight significant risks without any solid and clear use of strict methodologies, that shape their use across industry and professional practice.

The findings for the first research question show that AI and LLMs offer relevant benefits and enhancements to software development overall. The two most widely reported benefits in Table 4.4 indicate that AI tools can meaningfully accelerate development workflows and elevate output standards. Importantly, these advantages extend across the entire SDLC appearing not only in coding, but also in requirements and early design phases of the development overall, as well as in automated testing, debugging, and repair. This demonstrates that AI is not merely a code-generation tool, but a broad co-creative and analytical partner within software engineering practice.

With regard to the consequences of AI and LLMs usage in Software Development, the results of the second research question reveals that these benefits that were previously discovered are also inseparable from substantial and important risks. The three most frequently identified drawbacks within the articles in Table 4.5 directly undermine the foundational benefit of improved code quality. This introduces a central tension in current software engineering practice with AI and its mission, it promises acceleration and enhancement, but its integration and further use can generate new points of failure that are costly, unpredictable, and often difficult to detect. Legal, ethical, and intellectual property uncertainties are also connected to the previous risks since they further complicate adoption and incorporation. Moreover, the lack of standardized benchmarks and evaluation metrics makes it challenging to objectively assess the genuine contribution of AI tools to quality assurance and security within the SDLC, hindering accurate comparison and validation of their effectiveness.

The findings for the next research question which covers the Methodologies and Management Methods found within the literature show that current industry practice is not built around full automation at all, but around control. The most prevalent methods documented in Table 4.6 like In-Context Learning, Prompt Engineering and Chain-of-Thought Prompting all aim to shape and constrain AI behavior before code is generated. Meanwhile, the Human-In-The-Loop paradigm makes sure that developers remain directly responsible for reviewing, validating, and approving all outputs maintaining the human presence throughout the whole development process, ensuring a quality experience. Rather than replacing developers, AI currently functions as a tool that must be guided and supervised. The presence of multiple emerging frameworks, including Agile-based human-AI collaboration models mentioned in the articles, indicates that the field is actively searching for structured and repeatable integration patterns and methodologies, but no universally accepted approach has yet emerged.

This tension is reflected clearly in the fourth research question, which results are documented in Tables A.2 to A.4. Developers and practitioners express enthusiasm about productivity and efficiency gains, but are equally vocal in concerns regarding reliability issues, hallucinations and output variability with AI usage and outputs. Organizations and management echo this trade-off because while aiming to improve developer satisfaction and competitiveness, they prioritize human oversight during AI-assisted development, data security and privacy protection, often restricting or prohibiting the use of commercial

LLMs for confidentiality reasons. Both groups converge on the consensus that clear expectations must be established and that AI systems should augment developer expertise while maintaining a controlled level of trust regarding its capabilities.

Finally, the last research question results reveal that adoption is not uniform. The sectors most actively integrating AI is Software Engineering, Software Development and Consulting, where experimentation and rapid iteration are strategically valuable and lower-risk and LLMs are used both as operational tools and as accelerators of knowledge transfer. In contrast, sectors characterized by regulatory sensitivity or safety-critical operations such as Finance, Banking, Transportation, Aerospace, and Healthcare, all documented in Table A.5, exhibit cautious and controlled adoption, often requiring stronger guarantees of interpretability, security, and auditability before deployment. This reinforces the conclusion that the maturity of LLM integration is influenced not only by technological capability, but also by the risk tolerance and domain-specific requirements of the adopting organization.

To address the need for a clearer mapping between AI capabilities and the Software Development Lifecycle, Table 4.7 synthesizes the findings of the SLR by outlining the primary impacts, benefits, and challenges of AI and LLM usage across individual SDLC phases.

Table 4.7: Impact of AI and LLMs across Software Development Lifecycle Phases

SDLC Phase	Main AI Contributions	Main Challenges
Requirements Analysis	Support for requirements elicitation, clarification, summarization, and stakeholder communication; rapid exploration of solution ideas through conversational interfaces.	Risk of ambiguity amplification, overconfidence in AI-generated requirements, and lack of domain context; increased need for human validation early in the process.
Design	Generation of architectural alternatives, design patterns, API suggestions, and exploratory prototypes.	Limited understanding of system-wide constraints; risk of inconsistent or suboptimal architectural decisions without expert oversight.
Implementation	Code generation, refactoring, boilerplate reduction, and rapid prototyping; strong association with practices such as Vibe Coding.	Hallucinated code, inconsistent outputs, security vulnerabilities, and reduced trust due to output variability; heavy reliance on human review.
Testing	Automated test case generation, test data creation, bug localization, and support for exploratory testing.	Difficulty validating correctness of AI-generated tests; increased testing complexity due to unpredictable AI-generated code.
Deployment	Support for configuration scripts, infrastructure-as-code templates, and documentation generation.	Risk of misconfiguration or insecure defaults; limited contextual awareness of operational environments.
Maintenance	Assistance with debugging, documentation updates, code comprehension, and technical debt identification.	Potential over-reliance on AI explanations; challenges in maintaining long-term system understanding and accountability.

Overall, the SLR provides a comprehensive understanding of the current state of AI-driven software

development. The findings demonstrate that while AI and LLMs offer meaningful benefits, the absence of a robust, standardized, and secure framework and/or methodologies for integrating these tools into the Software Development Lifecycle remains the largest barrier to widespread adoption. The promise and current new "Vibe Coding" paradigm is clear, trying to make software development as high-level conceptual expressions rather than manual syntax construction, but realizing this vision requires overcoming significant challenges in reliability, safety, organizational governance and much more. Thus, the future of AI in software development depends not on improving the raw capability of models alone, but on creating methodologically sound, transparent, and trust-centered processes and workflows for using them effectively.

5

Research Problem

This chapter presents the first activity of the DSRM, the identification of the research problem and its motivation. The second activity, the definition of objectives, is presented together with the design of the artefact that satisfies them in Chapter 6, since the objectives derived here are what that design is answerable to.

The review reported in Chapter 4 shows that AI and LLMs are being adopted across sectors and across every phase of the SDLC, and that the field is evolving fast enough for new models and tools to appear continuously. It also shows that the benefits are accompanied by challenges that obstruct successful implementation. The most consequential of these, and the one reported most consistently, is the absence of established, clear methodologies shaping the proper use of these tools in professional software development [13–16].

Several frameworks and approaches that incorporate AI into the development process have been proposed, and are summarised in Table 4.6. They differ significantly in scope, assumptions and level of abstraction, and share no common foundation regarding lifecycle integration, management practice or evaluation criteria [3]. Organisations therefore face difficulty not only in adopting AI-supported practice but in assessing its impact on software quality, security and maintainability, because no standardised benchmarks or performance metrics exist against which to do so. The field is actively searching for structured and repeatable integration patterns rather than converging on them.

Established methodologies do not supply those patterns. Waterfall, Scrum, Kanban and the Agile Manifesto introduced substantial improvements in adaptability and collaboration, but were designed for environments in which every contributor is human [4]. In AI-driven contexts they fall short in three specific ways. High-speed AI-assisted coding accelerates implementation while creating pressure upstream, in requirements clarification, and downstream, in testing, verification and maintenance. They provide no explicit structure for governing AI participation, for assessing model reliability, or for mitigating the

resulting risks. And they assume all contributors are human, whereas a model is a new category of collaborator whose output carries no author and therefore no obvious owner.

The empirical case for treating this as a process problem rather than a capability problem is set out in Chapter 1 and is not restated here. Two independently reported findings carry it. Security performance does not improve as functional correctness improves, so generated code that works is not thereby safe [9]; and acceleration at the point of authorship relocates to human review rather than disappearing, without reaching delivery outcomes [12]. Both sources are industry-reported rather than peer-reviewed and are cited as such. Their value lies in agreeing on the mechanism rather than in the precision of any single figure: neither security nor delivery improves as a by-product of faster generation, and both depend on where validation and review sit in the process.

The core research problem addressed in this work is therefore the lack of patterned guidance, methodological rigour and standardised evaluation metrics for the systematic integration and management of AI and LLM-based tools across the SDLC, while preserving human oversight, software quality and security, and organisational accountability. This gap limits the ability of organisations to realise the benefits of AI-assisted software development sustainably.

The objectives a solution must satisfy, and the design that answers them, are presented in Chapter 6.

6

Research Proposal

Contents

This chapter presents the second and third activities of the DSRM: the definition of objectives for a solution to the research problem formalised in Chapter 5, and the design of the artefact that satisfies them, a process-oriented framework for governed Human-AI collaboration across the SDLC. The framework is described here as a methodology, independently of any implementation; its instantiation as a working system is the subject of Chapter 7.

6.1 Objectives

The systematic literature review in Chapter 4 established the current state of AI and LLM adoption in software development, and the analysis of its benefits and consequences led to the problem formalised in Chapter 5: the lack of structured guidance, methodological rigour and standardised evaluation practice for integrating these tools into development processes. The research objectives are formulated to address that problem directly, and Table 6.1 links each identified gap to the framework objective that resolves it.

Table 6.1: Mapping between Research Questions, Key Findings, Problem Aspects, and Framework Objectives

RQ	Key Finding from SLR	Problem Aspect Addressed	Resulting Framework Objective
RQ1	Usage is unevenly distributed across SDLC phases and lacks structured integration.	No structured guidance on where and how AI should be applied across the SDLC.	Define a process framework mapping AI-supported activities to specific phases.
RQ2	Recurring risks in reliability, validation, security and insufficient human oversight.	Insufficient methodological support for risk, quality and accountability.	Integrate governance mechanisms, human-in-the-loop practice and quality gates.
RQ3	Significant variation among approaches and no consensus on managing adoption.	Fragmentation of existing AI adoption methodologies.	Consolidate practice into a unified, repeatable process-oriented framework.

These objectives translate observed gaps into concrete design goals: mapping AI usage to SDLC phases, introducing governance and human-in-the-loop mechanisms, and consolidating fragmented practice into a unified process model. The remainder of this chapter presents the framework designed to meet them.

6.2 Purpose and Scope

The artefact is a method in the sense the DSRM gives that term: phases, hats, artefacts and decision rules prescribing how a team organises the participation of AI across the SDLC. It is stated at the level of process rather than of tooling, so that it can be assessed independently of the system instantiating it in Chapter 7 and adopted against a different tool stack. It covers AI-assisted activity in every phase rather than the code-authoring phases alone, the collaboration and model-selection practices that activity requires, expressed as process obligations rather than a catalogue of prompts, and the governance mechanisms of Section 6.6.

It excludes the training or fine-tuning of LLMs, model architecture design, Machine Learning Operations (MLOps) pipelines and tool-specific detail. These are boundaries rather than omissions and follow from the problem in Chapter 5, which concerns teams that consume general-purpose models to build software, not teams that produce models. MLOps governs the lifecycle of a model as a deployed asset; this framework governs the lifecycle of software a model helped write. An MLOps pipeline can

be entirely mature while the surrounding process still has no mechanism for deciding whether generated code answers the requirement it was derived from, which is the deficiency reported in Chapter 4. Tool-specific detail is excluded methodologically: a framework prescribing a vendor cannot be evaluated separately from that vendor, confounding the assessment of the method with that of the instantiation, which Chapter 7 and Chapter 9 keep apart.

6.3 Design Principles

Eight principles govern every subsequent design decision. Each is stated in Table 6.2 together with the evidence that motivates it, since a principle asserted without grounding is a preference, and a framework built out of preferences cannot be defended as a research contribution. Where the grounding is a design commitment rather than an empirical finding, that is said so.

Table 6.2: The eight design principles and the evidence grounding each

Principle	Statement and grounding
Human-in-the-Loop by Design	Every AI-generated artefact is subject to explicit human review and acceptance before entering the product. Adoption and trust diverge, at around 90% daily use against roughly 30% reporting low or no trust [7], and benchmarking shows the distrust warranted: the best agent and model combination was functionally correct 61% of the time but secure only 10.5% [8], and 45% of samples across more than a hundred models introduced an OWASP Top 10 vulnerability, with security flat as correctness improved [9].
AI as Augmentation, Not Substitution	Humans retain responsibility for technical, architectural and quality decisions. In a randomised controlled trial developers took 19% longer with assistance while estimating they had been 20% faster [11]; substituting judgement removes the only faculty able to notice such a discrepancy, and the SLR reports reduced code understanding as a recurring consequence of unsupervised delegation [3, 4].
Process-Level Governance of AI Usage	AI participation is governed through repeatable mechanisms making contributions visible, traceable and attributable. AI amplifies: capable teams become measurably more capable, dysfunctional teams measurably less so [7]. If the surrounding process sets the sign of the effect, governance must live in the process rather than in prompt discipline.
SDLC-Wide and Phase-Aware Integration	Participation is defined per phase, with hats and controls adapted to that phase's risks. Usage is unevenly distributed and concentrated on code generation, leaving requirements, design and maintenance with little guidance [3–5]; governing only the well-served phases would reproduce the gap the framework exists to close.
Hats, Not People	Every role named is a function that must be performed, not a person who must exist. A design commitment rather than an empirical claim, argued in Section 6.5; its visible consequence is that no role is named whose only content is oversight.
Explicit Responsibility and Accountability	For each AI-assisted activity a named hat is accountable for initiation, validation and acceptance. When an artefact has no identifiable author no party is answerable for its defects, and ownership is among the concerns most frequently raised by practitioners and organisations [3, 13]. Accountability attaches to a function so that it survives the absence of any individual.
Risk-Proportional Validation and Control	Validation rigour is proportional to potential technical, organisational and ethical risk. The defect profile is not uniform: vulnerability rates reach 72% for Java against 45% overall and cluster in specific weakness categories [8, 9]. A uniform control is either too costly for the routine case or too weak for the dangerous one, which is why remediation separates a Fast Lane from a Secure Lane.

Principle	Statement and grounding
Spec-Anchored Continuity	A persistent versioned specification is maintained alongside the codebase and is what every AI-assisted activity is constrained by and validated against. As established in Section 3.5 this instantiates spec-driven development rather than inventing it [40, 41]. The limitation belongs with the claim: its own principal source describes the practice as emerging and its scaling behaviour as an open risk [40], which Chapter 9 treats as something to test rather than assume.

6.4 Lifecycle Phases and the AI Interaction Mapping

This section answers DRQ1. The framework organises the lifecycle into six phases, preceded by a Strategic Alignment stage that sits outside the lifecycle proper because it is performed before the Squad is engaged at all: it decides whether a Unit of Work should exist, whereas the six phases decide how one is built. The six are Discovery and Requirements; Design, Prototyping and Architecture; Implementation; Testing; Deployment; and Maintenance. Table 6.3 fixes, for the stage and each of them, what enters, what the model is permitted to produce, what leaves, which hat is accountable, and which control must be satisfied before exit. It is the central artefact of this chapter and is not restated in prose here; what follows argues the two points on which the model departs from its closest published precedent, and then why the order of the phases is not the shape of the process.

Two mechanisms the table names are defined here, since both recur throughout the chapter. The **Interactive Bridge** is the interaction pattern governing Discovery and Requirements: the model drafts scenarios in plain natural language for human critique, the Trio confirms, adds, deletes or modifies them, and only the approved result is compiled into formal Gherkin and anchored. Separating critique from compilation matters because natural language is the medium in which a domain expert can detect a wrong assumption, whereas formal syntax invites review of the syntax instead. The **Consistency Factor** is the strict automated test set produced before generation begins, derived from both the functional and the technical specification, and it constrains what the model may produce rather than assessing it afterwards; it is specified in full in Section 6.6.1.

The framework's most direct precedent is the AI-Driven Development Lifecycle (AI-DLC) published by Amazon Web Services [42], which organises work into Inception, Construction and Operations, replaces sprints with Bolts, short cycles measured in hours or days, replaces Epics with Units of Work, and governs AI participation through collaborative rituals of which Mob Elaboration is adopted here. Those three terms are taken directly from AI-DLC with the same meaning and no originality is claimed for them. Its evidential status must be stated with equal directness: its reported outcomes, of the order of a ten to fifteen fold productivity improvement, originate from the vendor's own blog and conference material, have not been independently replicated or peer reviewed, and carry no evidential weight here. It is adopted as process vocabulary, not as evidence of a result. Most of the expansion from three phases to six is decomposition rather than disagreement, since Discovery and Requirements together with Design, Prototyping and Architecture occupy the territory it calls Inception, and Implementation corresponds to Construction. Two extensions are substantive.

The first is a decoupled Testing phase. In AI-DLC, build and test close Construction, performed in the same cycle and by the same participants that generated the code. The objection is not thoroughness but

circularity: tests derived by the same model, from the same specification, in the same context encode the same reading of the requirement the implementation encodes, so passing them confirms internal consistency rather than external correctness. The benchmark evidence shows exactly this asymmetry, since agent-generated code passes functional tests far more often than independently held security tests and the gap does not close as models improve [8, 9]. Testing therefore becomes its own phase with its own environment, hat and gate, working from the locked functional specification rather than the implementation. The unit of validation changes at this boundary: Bolts validate a task, Testing validates an assembled User Story, the smallest unit at which business value is observable.

The second is a distinct Maintenance phase with a diagnostic firewall. AI-DLC's Operations phase is deployment with monitoring and defines no mechanism for classifying post-deployment signals or routing them to an owner. The signal is heterogeneous: a report may be a defect against an agreed specification, a request for behaviour never specified, or evidence that the specification was wrong. Treating all three as defects erodes the specification, because each undocumented accommodation widens the distance between what the system does and what the record says it should. A signal is therefore classified before any code is opened, and Context Reconstruction requires the developer to supply the model with the specific report, the test evidence and the isolated fragment implicated, prohibiting whole-project context. Narrowing the context is a control, for the reason developed in Section 6.6.2 [8].

The phases are listed in an order, and it would be a misreading to take that order as the shape of the process; a reviewer of an earlier version raised precisely this objection, and the answer is structural. Maintenance neither terminates the lifecycle nor patches in place. Its exit is always an entry into a named earlier phase, whose gate then applies in full: a business deviation routes to the Project Manager (PM) hat for a value decision and, if approved, re-opens Discovery and Requirements; a specification gap re-opens Design, Prototyping and Architecture, and the technical specification is amended and re-versioned; a production defect re-enters Implementation as a Fix-Bolt and returns through Testing. The distinction from a Waterfall lifecycle with a maintenance tail is not one of degree. In Waterfall, post-release feedback is measured against a specification treated as frozen, and the specification is the one thing the feedback cannot change; here the specification is exactly what is re-opened, re-approved and re-versioned, and code is regenerated from it rather than diverging from an unchanged one. What the framework refuses is the unnamed loop, the change applied to production code without a corresponding change to the specification it is supposed to satisfy, because that is how traceability is lost in practice.

Table 6.3: The six lifecycle phases and the preceding Strategic Alignment stage: permitted AI contribution, output, accountable hat, and the control that must be satisfied before each is exited. Each phase takes as input the output of the one above it.

Phase	Role of AI	Output	Human accountability	Mandatory control
Strategic Alignment (pre-lifecycle)	Assists market analysis; drafts candidate objectives and indicators	A Unit of Work carrying a stated business justification, Objective and Key Results (OKRs) and Key Performance Indicators (KPIs)	PM hat aligns the product vision with stakeholders and owns the justification	Stakeholder alignment and OKR sign-off before the Unit of Work reaches the Squad

Continued on next page

Phase	Role of AI	Output	Human accountability	Mandatory control
Discovery & Requirements	Drafts fractional User Stories in natural language; compiles approved scenarios into formal Gherkin	Approved User Stories sized XS or S, with Gherkin acceptance criteria and stable scenario identifiers	The Trio validates correctness, completeness and business value through the Interactive Bridge	Mob Elaboration and the Discovery Gate
Design, Prototyping & Architecture	Proposes architectural options; generates User Interface (UI) structure and a formal technical specification	Approved prototype and an anchored technical specification (interface contract, data model, runtime contract)	Design Lead hat validates usability; Tech Lead hat validates feasibility, scalability and integration risk	Design Gate; no task enters the Bolt Backlog until both validations pass
Implementation	Generates and refactors code constrained by the Consistency Factor test set	Task-level code passing the pipeline and the Consistency Factor	Developer hat validates output, owns correctness and security, and must be able to explain the logic	Pre-Bolt architectural lock, the Bolt micro-cycle, and the Implementation Gate at story level
Testing	Drafts end-to-end Behaviour-Driven Development (BDD) scripts derived from the approved Gherkin	A story-level verdict and, on failure, the exact failing evidence	Quality Assurance (QA) hat verifies and refines generated scripts before execution and probes edge cases manually	The QA Validation Playbook and the Testing Gate, in a separate environment decoupled from the Bolt
Deployment	Answers the infrastructure delta question; generates configuration, Infrastructure as Code (IaC) or migrations only when required	A released increment and a persistent context updated to the live state	Whoever performs the deployment approves it and reviews its security implications	Deployment Gate, performed by the deploying hat; no separate gatekeeping team is assumed
Maintenance	Reconstructs a deliberately narrow context to explain one specific failure and proposes a bounded fix	A classified signal routed to the phase that owns it, with a verified diagnosis where applicable	Developer or PM hat classifies the signal and must verify any diagnosis before it is trusted	Diagnostic firewall (Context Reconstruction, whole-project context prohibited) and governed loop-back to a named phase

6.5 Roles and Responsibilities

Every role named is a hat: a function the framework requires to be performed, not a person who must be hired. One individual may wear several hats, and on a small team a hat may go unfilled, its responsibility falling to whoever performs the adjacent work. Table 6.4 summarises the hats and the gate each owns. Expressing responsibility as a function rather than a position is a design decision made for three reasons. The framework targets anyone with a technical background, working alone or in a small team, and a role model presuming a staffed organisation would exclude precisely the population that has adopted AI assistance fastest. Accountability expressed as a title lapses the moment the title is vacant, whereas accountability expressed as a function survives reassignment. The third reason is evidential, and concerns what happens when structural models are copied as blueprints.

The Trio, comprising the Product Owner (PO), Tech Lead and Design Lead hats, is inspired by the Squad concept popularised in the Spotify Model, and citing that model honestly requires citing its limits alongside it. The Squad, Tribe, Chapter and Guild structure was published as a description of how

one company was organised at one point in time, not as a method offered for adoption elsewhere [43], and its principal author has since been reported as treating it as inspiration rather than a blueprint [44]. A first-hand account by a former employee argues it was never fully implemented even internally and names two failure modes: a matrix structure that left decision authority unclear, and autonomy without corresponding accountability [45]. Those two are why the framework's people structure carries governance the original did not. Matrix ambiguity is answered by the Tech Lead hat's architectural lock, which fixes the technical boundary before any Bolt begins so that technical authority is exercised at a defined moment rather than negotiated continuously; autonomy without accountability is answered by separating logistics authority in the Bolt Master hat from technical authority in the Tech Lead hat, and by attaching every gate to a named function. The claim is not that the framework implements the Spotify Model but that it is inspired by it and hardens against the ambiguity its own critics identified. A more rigorous vocabulary exists for the same argument: the stream-aligned team, owning a business capability end to end, is a closer structural match than the Squad, and treating team cognitive load as an explicit design constraint [46] explains why the framework limits developer agency over scope during a Bolt, since reducing the open decisions a developer must hold while validating generated code is itself a cognitive-load reduction, and validation is the activity Chapter 1 identifies as the new constraint. The PM hat sits outside the Squad, structurally closer to the single-threaded leader model that Amazon adopted after its two-pizza structure did not survive scale [47]; that history is cited because even the companies whose structures are most often reproduced as blueprints describe them as provisional.

An earlier version named a DevOps Alliance, an external group providing oversight during deployment and holding two responsibilities: approving the release and reviewing the security of generated infrastructure code. It has been removed with no replacement, because it named a governance function without naming who holds it, which is the defect the principle of explicit responsibility exists to prevent: an oversight body defined only by what it supervises can be assumed to exist by the process and never staffed by the organisation, and the gate is then satisfied by whoever is available while appearing to have been satisfied by a specialist. Both responsibilities remain mandatory and only their owner changed, being performed by whoever carries out the deployment, self-reviewed on a team of one and peer-reviewed where a second developer exists. The framework prefers a control whose owner is unambiguous and occasionally the same person to one whose owner is impressive and frequently fictional.

The gates do not relax as the team shrinks. On a solo project each gate is self-checked, but what it requires is unchanged: the evidence in Table 6.5 must exist, be recorded, and be examined deliberately at the moment the gate is reached rather than assumed to have been examined during the work that produced it. The value of a gate lies primarily in forcing an explicit, separate act of validation against a written specification and only secondarily in who performs it. Where none is available the gate is a checkpoint rather than a handover, and the persistent specification is what makes self-review meaningful, because the reviewer checks the artefact against a record written before it existed.

Table 6.4: The hats: the function each discharges and the gate each owns

Hat	Function the framework requires	Gate owned
PM	Stakeholder alignment, OKRs and KPIs, and the business justification of a Unit of Work before it reaches the Squad; receives classified business deviations from Maintenance.	Strategic alignment sign-off

Hat	Function the framework requires	Gate owned
PO (Trio)	Business value and prioritisation of User Stories; leads the critique of AI-drafted scenarios during Mob Elaboration.	Discovery, with the Trio
Tech Lead (Trio)	Technical specification, architectural feasibility and integration risk; decomposes stories into tasks and flags high-risk ones before any Bolt begins.	Design (technical) and the pre-Bolt architectural lock
Design Lead (Trio)	Prototype or visual element for every feature, and its validation through usability testing.	Design (usability)
Bolt Master	Continuous flow: prioritises the Bolt Backlog, manages dependencies, assigns by capacity once the Tech Lead hat has approved.	None; holds logistics authority deliberately separated from technical authority
Developer	Executes tasks with the model, generates and validates the Consistency Factor, and remains answerable for correctness, security and being able to explain the logic.	Implementation
QA	Validates the assembled story against the approved Gherkin in a separate environment, verifying generated BDD scripts before execution and probing edge cases manually.	Testing
Deploying hat	Not a distinct role: whoever performs a release approves it and reviews the security of any generated IaC or configuration.	Deployment

6.6 Governance Mechanisms and Quality Gates

6.6.1 Bolts as the Unit of Execution

A Bolt is the smallest execution iteration the framework recognises: the implementation of one task within a User Story, strictly inside the development environment and measured in hours rather than weeks. The term is adopted from AI-DLC [42]; what the framework adds is the sequence of obligations surrounding it, since a short cycle without a fixed reference is a fast way to accumulate divergence. Before any code is generated the Tech Lead hat feeds the locked context, comprising the approved Gherkin, the prototype and the technical specification, to the model, verifies the task list it proposes and flags high-risk tasks; this architectural lock makes the technical boundary a matter of record rather than negotiation, which is what allows tasks on either side of an interface to proceed in parallel. The Bolt Master hat then assigns by capacity, giving developers the story's intent so they can recognise a wrong answer but not authority over technical scope, which the lock has settled. A Developer hat pulls one task, at which moment the Bolt begins and Bolt Cycle Time is measured from. Before generation, developer and model together produce the Consistency Factor, a strict automated test set derived from both specifications whose purpose is to constrain generation rather than report on it afterwards, fixing the shape of the responses the implementation is permitted to produce. The model generates against that constraint and the developer refines the result as an active editor rather than accepting or rejecting a finished proposal. A task is done when the code passes the pipeline and the developer can explain the logic; the second condition is the operational form of the augmentation principle, since code no one can explain has been substituted for engineering rather than added to it. When every task is done the story passes the Implementation Gate into staging. The Fix-Bolt is the single exception: triggered by a failed Testing Gate or a verified defect, it omits task breakdown because the context already contains what that

step would produce, namely the exact failing scenario and the evidence of its failure. It is justified by what is already known and is not a licence to skip planning when a change feels small. Its outcome is routed by risk, which is where the risk-proportional principle becomes operational: a low-risk patch takes the **Fast Lane** directly to the delivery pipeline without re-entering formal QA, while a higher-risk one takes the **Secure Lane**, returning to staging for a Regression Bypass in which QA re-runs the existing suite before verifying the targeted patch. Two lanes exist because a single uniform level of control is either too costly for the routine case or too weak for the dangerous one.

6.6.2 Spec-Anchored Continuity

Spec-Anchored Continuity requires every AI-generated artefact to carry forward the context established by previously validated phases. The mechanism is a persistent versioned specification maintained alongside the codebase and split into two layers with different lifetimes. The **global layer**, the Memory Bank, holds what does not change between stories, namely architectural rules, technology-stack decisions, security policies and recorded workarounds for already-diagnosed defects; it is written rarely, amended deliberately, and applies to every generation on the project. The **local layer**, the Feature Specs, holds what is specific to the work in hand, namely the approved Gherkin acceptance criteria and the technical specification produced at the Design Gate; it is written once per story, locked at a gate, and amended only through a recorded versioned amendment. The split is what makes the mechanism usable, since a single undifferentiated context grows monotonically until it contains more irrelevant material than relevant, reintroducing the problem it was meant to solve.

The central argument is that narrowing the model's working context is a control rather than a convenience, and the distinction is not semantic: a convenience makes a correct outcome easier to reach, whereas a control is a measure whose absence makes an incorrect outcome undetectable. Anchoring satisfies the second definition on three counts. It supplies a reference that existed before the generated artefact did, which is the precondition for validation being possible at all, since without it a reviewer can assess only whether generated code is internally coherent and not whether it answers the requirement. It bounds what the model may act upon, an unbounded context being the condition under which a model produces a wide, confident and substantively wrong change, which is why whole-project context is prohibited during Maintenance diagnosis [8]. And because both layers are versioned and every post-lock edit is a dated amendment, divergence between what was agreed and what was built is a detectable event rather than an unattributable drift. As established in Section 3.5 this instantiates spec-driven development rather than inventing it [40], and recent work independently argues, from a multivocal review, that specification discipline rather than model capability is the binding constraint on dependability; that work is a preprint and is cited as supporting evidence rather than settled consensus [41]. The contribution is not the idea of anchoring but the specification of where it is mandatory, which gate locks each layer, and what constitutes a legitimate amendment.

6.6.3 Quality Gates

A quality gate has three named components: an owning hat, a specific body of evidence that must exist and be examined, and a defined consequence when it is absent or inadequate. A checkpoint missing any of the three is a ceremony rather than a control, since it can be passed without anything being

established. Table 6.5 specifies the five gates on those terms. Two properties are worth stating. Failure never discards work and never silently degrades it; it returns the work to a named earlier activity with the evidence attached, which is the same routing discipline Maintenance applies to post-deployment signals. And every gate is defined against recorded evidence rather than a judgement of readiness, which is what allows it to remain meaningful when self-checked by a solo practitioner: the practitioner is not certifying their own confidence but confirming that a specific artefact exists and matches a specification written before it.

Table 6.5: The five quality gates: owner, required evidence, and the consequence of failure

Gate	Owning hat	Evidence required to pass	Consequence of failure
Discovery	The Trio	Testable Gherkin acceptance criteria approved through the Interactive Bridge; every story sized XS or S; the functional specification locked and versioned.	Scenarios return to Mob Elaboration; oversized stories are decomposed before re-submission.
Design	Design Lead and Tech Lead hats	An approved prototype validated by usability testing; a technical specification covering interface contract, data model and runtime contract, anchored into the persistent context.	Feedback returns to prototype generation or architectural specification; no task may enter the Bolt Backlog.
Implementation	Developer hat	Every task complete, passing the pipeline and the Consistency Factor, with the developer able to explain the generated logic.	The story does not enter staging; incomplete or unexplained tasks return to the Bolt Backlog.
Testing	QA hat	The assembled story validated in staging against the locked Gherkin through verified BDD scripts, with edge cases probed manually.	A Fix-Bolt is triggered with the exact failing evidence attached; the story re-enters staging afterwards.
Deployment	Whoever performs the deployment	Explicit approval of the release and a security review of any generated IaC, configuration or migration; the persistent context updated to the live state.	The specific objection is fed back for a compliant regeneration; the release does not proceed.

6.6.4 Governance Metrics

Three governance metrics are defined, chosen to measure whether the collaboration is safe and whether it is flowing rather than to measure output volume, since output volume is the quantity Chapter 1 shows rising without a corresponding improvement in delivery [12]. **Bolt Cycle Time** is the elapsed time from a task being pulled as a Bolt to its being marked done, defined at task level rather than story level because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it, and the framework's claim about continuous flow is a claim about execution. **Context Traceability Rate** is the proportion of deployed Bolts and stories whose context chain is complete, meaning the deployed artefact resolves back to a locked technical specification, to the functional specification it derived from, and to the Unit of Work that motivated it; it measures whether Spec-Anchored Continuity is holding in practice as opposed to being prescribed. **AI Defect Escape Rate** is the proportion of production defects originating in AI-generated logic that passed the Testing Gate, and it is the metric that detects a gate which has become ceremonial, since a gate satisfied without being exercised shows up here and

nowhere else.

These are expressed in vocabulary continuous with established AI governance standards rather than bespoke to this work. The National Institute of Standards and Technology (NIST) AI Risk Management Framework organises governance into Govern, Map, Measure and Manage [48], corresponding respectively to this chapter's design principles, phase mapping, metrics and gates, and ISO/IEC 42001:2023 offers a complementary plan-do-check-act vocabulary describing the same loop [49]. This alignment is a claim about shared language and nothing more: neither the framework nor its instantiation has been audited against either standard, no conformity assessment has been performed, and nothing here should be read as asserting compliance with, certification against, or the intention to certify against either document. The purpose is narrower, namely to show that these constructs occupy the conceptual space standards bodies are converging on, so that a team already working in that vocabulary can locate the framework's mechanisms within it.

6.7 Work Organisation

The framework replaces the familiar hierarchy of epics, sprints and sprint backlogs with continuous flow. That is the most disruptive single element of the proposal and therefore requires the strongest justification, which is empirical rather than stylistic. Sprint cadence is a mechanism for managing a world in which implementation is the binding constraint: a fixed-length iteration exists so that a quantity of implementation work can be forecast, committed to and inspected at a boundary, and its length is calibrated to how long a useful amount of implementation takes. The evidence in Chapter 1 indicates that premise no longer holds where AI assistance is heavily used. In a randomised controlled trial, experienced developers working on repositories they knew well took 19% longer with assistance while believing they had been faster, so the effect is not simply a reduction of implementation duration that can be absorbed by planning for a shorter one [11]. At organisational scale, telemetry from more than ten thousand developers across 1,255 teams associates high adoption with 98% more pull requests merged and 154% larger pull requests, but also with 91% longer review times and no significant correlation with company-level delivery outcomes [12]. The consistent reading is that the constraint moved rather than disappeared: change volume rose, review lengthened, delivery stayed flat. A cadence calibrated to a constraint that has moved does not merely fail to help, it obscures the new constraint, because the iteration boundary continues to report on the activity that is no longer limiting. The structural response has two parts: execution is compressed into Bolts so that the queue in front of validation is refilled in small increments rather than batches, and validation is decoupled and given its own phase, environment, hat and gate so that the constraint is scheduled and measured in its own right. Both are falsifiable: if review is not the constraint in a given setting, Bolt Cycle Time and the QA queue will say so.

Three levels of work are retained, and the vocabulary of the upper two follows AI-DLC [42]. A **Unit of Work** is the core business problem, owned by the PM and PO hats, and is the level at which business justification is argued and acceptance criteria are generated as a group; generating them grouped this way rather than requirement by requirement is itself supported by evaluation, since across 107 requirements an epic-organised pipeline outperformed a requirement-aligned baseline on expert-rated correctness, 4.61 against 4.14 out of 5, and completeness, 4.31 against 3.50 [50]. **User Stories** are the tangible deliverables, constrained to extra-small or small, and are the level at which QA tests

because they are the smallest unit at which business value is observable; the size constraint is load-bearing rather than aspirational, since a story describable by a handful of scenarios is small enough for a reviewer to hold entirely in mind while validating generated code against it. **Tasks** are the granular technical steps executed as Bolts. The **Bolt Backlog** is a continuous prioritised queue that tasks enter only after decomposition with the model, technical verification by the Tech Lead hat, and risk flagging. Developers have limited agency over task creation and selection, which is a deliberate restriction rather than an expression of distrust: the decisions withheld are precisely those the architectural lock has already settled, and every decision left open during a Bolt is one more thing a developer must hold in working memory while performing the validation the framework depends on. Finally, the team's project management tool is expected to expose states corresponding to the phases; these names are guidance rather than a literal requirement, since teams already have boards and the framework's purpose is to reason over an existing board rather than replace it, so what matters is that each column maps unambiguously onto one phase.

6.8 Operational Playbooks

The phases, gates and hats state what must hold; the six playbooks in Table 6.6 state how the recurring procedures are performed, each with its trigger, its participants and its termination condition. Together they are the operational content of the framework: a team that follows these six and observes the gates in Table 6.5 is operating the framework, irrespective of the tooling it uses.

Two of the steps are substantive design choices rather than sequencing. In Maintenance, the prohibition on whole-project context is the single most restrictive rule in the framework, for the reason given in Section 6.4. In the Fix-Bolt, the Regression Bypass re-runs the existing BDD suite rather than regenerating it, because a regenerated suite is a different measurement instrument, and comparing a patched system against it cannot distinguish a fixed defect from a changed test.

Table 6.6: The six operational playbooks: trigger, procedure, and termination

Playbook	Trigger	Procedure	Terminates when
Mob Elaboration	A Unit of Work brought to the Squad	The Trio supplies intent, personas and domain constraints; the agent drafts fractional stories in natural language; the Trio cross-examines for value, feasibility and coherence, issuing confirm, add, delete or modify instructions until satisfied; the agent then compiles the approved scenarios into Gherkin and anchors the result.	The Discovery Gate passes and the functional specification is locked
Design and Prototyping	Locked Gherkin acceptance criteria	The Design Lead hat prompts for screens and flows; the Tech Lead hat prompts for interface contract, data model and runtime contract and anchors them; usability testing and architectural review run in parallel.	Both validations pass and tasks enter the Bolt Backlog

Playbook	Trigger	Procedure	Terminates when
Maintenance and Evolution	A post-deployment signal	Classify before any code is opened: a business deviation or specification gap routes to the PM hat for a value decision and re-opens the owning phase; a technical defect routes to a Developer hat, who narrows the context to the report, the test evidence and the isolated fragment, asks only why that logic failed, and verifies the diagnosis manually.	The signal is classified and routed, with a verified diagnosis where applicable
Fix-Bolt	A failed Testing Gate or a verified defect	Pulled immediately with task breakdown skipped; the agent resolves only the specific defect while adhering to the anchored specification; the developer reviews for newly introduced vulnerabilities, not only symptom resolution; existing and new unit tests must pass.	Low risk takes the Fast Lane to the pipeline; higher risk takes the Secure Lane through a Regression Bypass in staging
QA Validation	A story that has passed the Implementation Gate	The QA hat opens the locked functional specification rather than the implementation, prompts for end-to-end BDD scripts and verifies them manually before execution, since an unverified script can report a passing scenario it never exercised, then probes edge cases exploratively.	The Testing Gate passes, or a Fix-Bolt is triggered with the failing evidence
Deployment and Release	A story that has passed the Testing Gate	The agent answers one bounded question, whether the release requires new infrastructure, configuration or migrations, and generates artefacts only when it does; whoever deploys reviews any generated artefact for vulnerabilities and policy violations.	The Deployment Gate passes and the persistent context is updated to the live state

6.9 Positioning Against Alternatives

A design is only defensible against named alternatives. Four are engaged below, each stated at its strongest, credited with what it genuinely does better, and met with the evidence motivating a different choice. The framework's constructs are an original synthesis; the sources cited throughout ground individual pieces rather than forming a joint blueprint this work reproduces.

No framework at all. The premise is that model capability suffices and process overhead is pure cost. What it does better should be conceded: no coordination cost, no artefacts to maintain, no gate that can become ceremonial, and for a short-lived prototype with no maintenance horizon it is very likely correct. Its failure mode is documented rather than hypothetical, since the large majority of functionally correct agent output was found to be insecure and generated code carried substantially more vulnerabilities than human-written equivalents [8, 9]. The decisive detail is not the size of either figure but that security performance did not improve as models improved on functional correctness: if it were merely lagging capability, waiting would be a valid strategy, but it is flat, which turns validation from a matter of discipline into a structural requirement.

Adopt AI-DLC as published and unmodified. The premise is that a coherent three-phase model already exists from a credible source and that extending it adds complexity without control [42]. What it does better is significant: it is simpler, has been operated at commercial scale, and carries less process

surface for a small team. The objection is narrow rather than general, and is the two gaps argued in Section 6.4: build and test folded into the closing stage of Construction, and an Operations phase with no published mechanism for classifying post-deployment feedback. The divergence fills gaps its own documentation leaves open rather than reinventing what it already solves. It should also be recorded that its reported productivity outcomes are vendor-reported and unverified, so adopting it unmodified cannot currently be justified by evidence of results.

Keep Scrum or SAFe cadence and treat AI as a faster development environment. The premise is that existing ceremonies already provide planning, review and retrospection, and that AI changes the speed of one activity without changing the structure. What it does better is not trivial: the cadence is understood, tooled, compatible with organisational planning cycles, and costs nothing in retraining. The evidence against it is the strongest available on this point because it includes a controlled experiment rather than only survey data, and one detail bears directly on this alternative: developers estimated afterwards that assistance had made them substantially faster when measurement showed it had made them slower [11]. The retrospective is the ceremony this alternative relies on to notice that something has gone wrong, and it is a weak instrument for noticing precisely this, while an iteration boundary calibrated to implementation continues to report on implementation while the constraint sits in review [12].

Metrics-only governance, with no phase structure. The premise is that an organisation can track the right capabilities and outcomes, publish them, and let teams self-organise. This is the most serious of the four: it is far cheaper to adopt, presumes no particular lifecycle shape, adapts to differing contexts, and the multidimensional account of developer productivity on which it rests legitimately argues that no single metric can stand for the whole and that any measure elevated to a target ceases to be a good measure [51], a warning applying to this framework as much as any other. The response is a distinction rather than a rebuttal. Metrics measure whether governance is working; they are not themselves a governance mechanism. A dashboard can report accurately that review time is rising and defects are escaping, but it contains no mechanism determining what a model may act upon, no point at which an artefact must be checked against a specification written before it existed, and no rule about where a post-deployment signal is routed. The two are therefore complementary rather than exclusive, which is why the framework defines its own metrics in the same vocabulary rather than proposing an alternative to measurement.

6.10 Summary

DRQ1 asked how AI-supported activities can be mapped to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous. The answer is the six-phase model of Section 6.4 with the Strategic Alignment stage preceding it, and the mapping in Table 6.3, which fixes for each phase what enters, what the model may contribute, what leaves, which hat is accountable and which control must be satisfied before exit. The boundary is made unambiguous in two ways a phase list alone would not achieve: the model is constrained at every phase to the anchored specification for that phase rather than to the project as a whole, and every phase terminates in a human act of acceptance recorded against named evidence. The map extends its closest precedent at two points, a decoupled Testing phase and a Maintenance phase with a diagnostic firewall, both argued from evidence rather than announced.

DRQ2 asked which governance mechanisms, gates and human-in-the-loop practices keep AI con-

tributions traceable, validated and accountable. The answer is the set in Section 6.6: Bolts as the governed unit of execution, with the Consistency Factor constraining generation rather than assessing it afterwards; Spec-Anchored Continuity as a two-layer versioned specification making the narrowing of the model's context a control rather than a convenience; five gates each with a named owner, named evidence and a defined consequence on failure; and three metrics expressed in vocabulary continuous with established governance standards, without any claim of compliance. Accountability attaches to functions rather than positions so that it survives a team of one, and every failed gate routes work back to a named phase rather than discarding it.

Two limitations belong here rather than in a later concession: spec-driven development, on which Spec-Anchored Continuity rests, is characterised by its own principal source as an emerging practice whose scaling behaviour is unsettled [40], and the framework's constructs are an original synthesis argued from evidence but not yet demonstrated in use. Chapter 7 therefore presents Apex, the reference implementation, and reports which constructs were implementable as specified, which had to be weakened or proxied, and which resisted implementation altogether, which is the substance of DRQ3.

7

Apex: The Reference Implementation

Contents

The framework presented in Chapter 6 describes a methodology. This chapter presents Apex, the working software system that instantiates it, which serves both as evidence that the framework's governance constructs are implementable and as the vehicle through which the framework is demonstrated and evaluated in Chapters 8 and 9. In DSRM terms the two are distinct artefacts produced by the same Design and Development activity: a method and its instantiation. This chapter answers DRQ3, and answering it honestly means reporting which constructs resisted implementation as well as which did not.

7.1 Purpose and Positioning

Apex is not a coding agent and is not positioned against one. A coding agent accelerates the writing of code; Apex governs whether the code that results still answers the question that prompted it. The two are complementary, and the distinction is the whole reason the system exists: acceleration without anchoring

is precisely the condition Chapter 1 shows moves the bottleneck to review rather than removing it. Every artefact Apex produces carries a stable specification identifier back to the requirement it derives from, so divergence between what was agreed and what was built becomes a detectable event rather than a gradual drift that nobody is positioned to notice.

What Apex deliberately does not do is as much part of its definition. It does not train or fine-tune models, and treats the model as a third-party capability that may change beneath it. It does not provision infrastructure: where a release needs configuration or migration scripts it generates them for a human or a Continuous Integration (CI) pipeline to execute, and never applies them itself. And it does not replace the project management tool a team already uses. It sits above that tool, reads and writes through it, and expects the team's existing board to remain the system of record for work items, which is why the framework's board guidance in Section 6.7 asks only that a team's own columns map onto framework phases rather than that they be renamed.

7.2 Architecture

Apex is a split full-stack application. The frontend is Next.js 15 with the App Router in TypeScript; the backend is Python 3.12 with FastAPI; the two communicate over a typed REST API. Server state on the frontend is owned by React Query and client state by Zustand, with credentials persisted only to session storage so that they clear when the tab closes. Backend routers are kept deliberately thin, with workflow logic held in a service layer and a schema for every request and response boundary. That separation is an engineering choice rather than an accident of growth: it keeps route handlers testable as thin adapters and workflow logic testable in isolation from Hypertext Transfer Protocol (HTTP) concerns, which is what made the test strategy in Section 7.5 possible at the sizes it reaches. Figure 7.1 shows the arrangement.

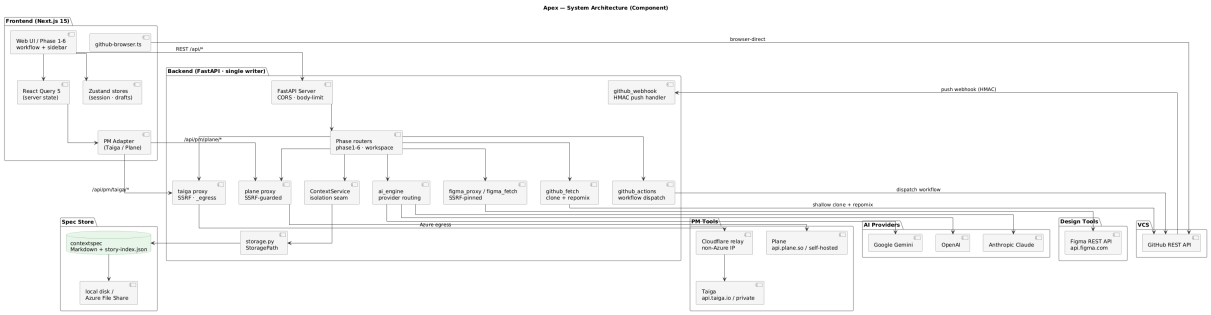

Figure 7.1: Apex's system architecture. The point of the figure is the position of the two boundaries rather than the inventory of components: every outbound call to a user-configurable host passes through a server-side guarded proxy, and all workflow state is written to a per-tenant, per-project specification store rather than to an application database.

Four properties of that architecture carry framework constructs rather than merely supporting them, and each is described in turn.

Spec-anchored storage. All workflow state lives in Markdown context files together with a machine-readable story index, held under a path keyed first by instance and then by project. The instance key isolates tenants, so a hosted project management account and a privately deployed instance of the

same tool are fully separate tenants even for the same human user; the project key isolates individual projects within a tenant. Storage is an abstraction over either local disk or cloud file storage, and the same code path runs in both, which the test suite verifies by pinning the abstraction to local disk rather than replacing it with a stand-in. This is Spec-Anchored Continuity made concrete: the specification is a set of files that outlive any session and that every generation step is constrained by. Access to those files is required to pass through a single service, never a direct import from a route, because that service is the one place where per-request project isolation is enforced. That convention is treated as an architectural invariant rather than a style preference, for reasons Section 7.5 makes concrete.

A deliberate single-writer concurrency model. The backend was originally pinned to one replica and one worker. The story index and workspace configuration live on shared file storage guarded only by a process-local lock, so a second replica would not crash but would silently race on writes, which is worse, because a lost update looks like success. Threads still interleave within the one process, so an in-process lock remains necessary and is used consistently. This was a documented constraint rather than an oversight, and it came with a stated precondition for lifting it. That precondition was subsequently built and enabled: an optional coordination layer promotes the locks to cross-replica equivalents when a coordination service is configured, and falls back to byte-identical single-replica behaviour when it is not. It has been running in production with three replicas since June 2026, verified by inspecting live client connections rather than by reading a startup log line.

Proxy-first external integration. Every outbound call to a host the user can configure is proxied server-side rather than issued from the browser, specifically to close server-side request forgery: the proxy validates that the target uses Hypertext Transfer Protocol Secure (HTTPS) and resolves to a non-private address before forwarding, and caps the size of any body it relays. The identity anchor a request is validated against is derived from the same validated host, which is what makes the storage isolation above trustworthy rather than merely tidy. One integration is a deliberate exception, since the provider permits direct browser access and the credential never reaches the backend for those calls; two operations within it are not exceptions, because they need a server-side filesystem and a server-held credential respectively.

Provider abstraction. The model provider is detected from the model identifier and routed accordingly, so no other part of the codebase branches on which vendor is in use, and provider errors are mapped to typed exceptions at a single boundary so that every route inherits consistent semantics without repeating the mapping. This is what allows the framework's claim in Section 6.2, that it names the property a mechanism must have rather than the product providing it, to hold in the instantiation as well as in the method.

7.3 Operationalising the Framework

This section answers DRQ3, which asks how far the framework's governance constructs can be operationalised in a working system and which of them resist it. Table 7.1 states the mapping construct by construct, and every row points at something that exists in the codebase rather than something planned. Three outcomes are distinguished and the distinction is the substance of the answer: a construct realised as specified, a construct realised as a proxy that measures something adjacent to what the framework defines, and a construct that resisted implementation and is not present. Figure 7.2 shows

how a practitioner actually walks the phases those constructs govern.

Table 7.1: Framework constructs against the mechanisms implementing them in Apex, and the outcome of each

Construct	Implemented mechanism	Outcome
The six lifecycle phases	One workflow surface per phase, each gated on the preceding phase's exit condition rather than freely navigable.	Realised
Quality gates	A per-story phase-status flow advancing through gherkin_locked, design_locked, implementation, qa, qa_passed and deployed; middleware refuses phase access when the status does not permit it.	Realised
Spec-Anchored Continuity	Versioned Markdown context files plus a story index under a per-tenant, per-project path, split into project-wide files and per-story specifications.	Realised
Stable specification identifiers	Epic, entity, screen and scenario identifiers minted at generation time and recorded in a specification index, so every later artefact resolves back to a requirement.	Realised
The Interactive Bridge	Phase 1 drafts scenarios in natural language for confirm, add, delete or modify instructions, and compiles to Gherkin only after approval.	Realised
The Consistency Factor	A generated test set produced before implementation and carried into the developer pack for each task.	Realised
Bolts and the Fix-Bolt	A per-task micro-cycle recording pack_ready, pushed and done transitions with timestamps, independent of the story's own phase status.	Realised
Bolt Cycle Time	Elapsed time from pack_ready to done per task, aggregated as median and 90th percentile. Initially computed from story-level gate transitions and presented under the same name, which measured queueing and review rather than execution; corrected to the task-level definition the framework specifies.	Realised, after a proxy was found and replaced
Context Traceability Rate	Not implemented. The traceability graph exists and a chain can be inspected per story, but no aggregate proportion of deployed work with a complete chain is computed.	Resisted
AI Defect Escape Rate	Not implemented. It requires production defect attribution to generated logic, which the system has no mechanism to observe.	Resisted
Spec-drift detection	Built and then removed. Because the technical specification was a single shared file per project, an edit could not be attributed to the stories it actually affected, so any change flagged every story at or past design lock; one real edit flagged 44 stories at once. The amendment and versioning audit trail was kept.	Resisted at the granularity the framework assumes
Server-side reconciliation of generated output	Coverage claims and task dependency identifiers are re-checked against the parsed specification rather than trusted as reported, with unmatched claims dropped.	Realised, and added because it was needed

Three of those rows deserve emphasis because they are the ones that answer DRQ3 rather than merely illustrating it.

The **Bolt Cycle Time** row is the clearest instance of a proxy being mistaken for the construct. The framework defines the metric at task level precisely because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it. The first implementation measured story-level gate transitions and displayed the result under the framework's own name for the task-level quantity, which is exactly the substitution the framework warns against elsewhere. It was corrected, but

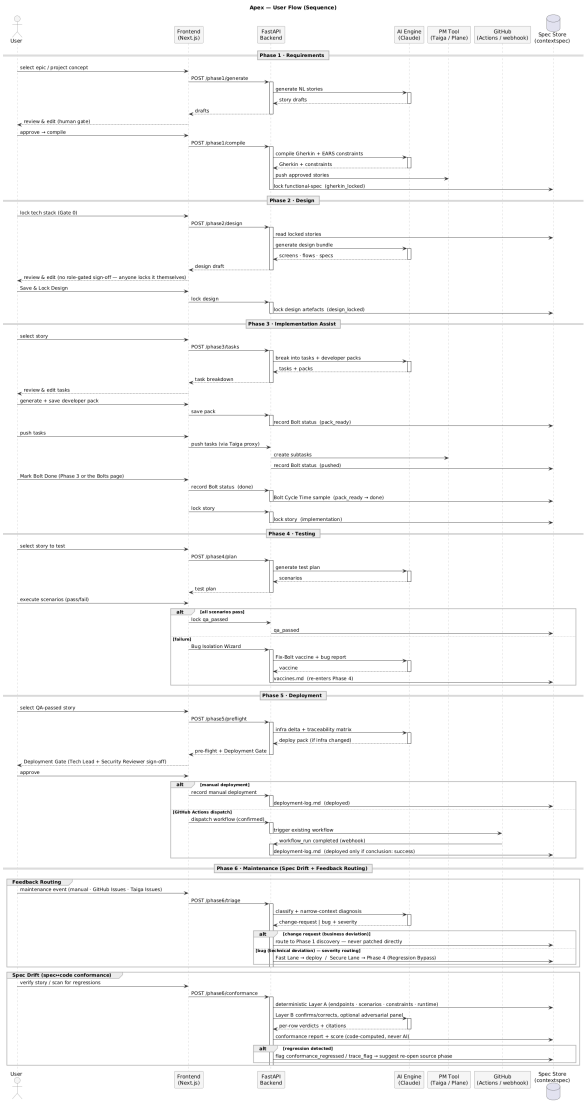

Figure 7.2: A representative walk through Apex phase by phase. What the figure is meant to show is that every phase terminates in a write to the specification store and a human acceptance, so no phase can be exited by generation alone.

the fact that it happened at all is evidence for a general point: a construct can be implemented, displayed and believed while measuring something else, and only comparing the implementation against the written definition catches it.

The two **unimplemented metrics** are reported as unimplemented rather than described as future work. Context Traceability Rate is computable in principle, since the chain it would measure is present per story, and its absence is a gap in the instantiation rather than in the method. AI Defect Escape Rate is different in kind: it requires attributing a production defect to generated logic, and nothing in the system observes production defects at all. That is a limitation of what a tool sitting above a project management board can see, and it is the strongest single piece of evidence that not every construct in Section 6.6.4 is instantiable by this class of system.

Spec-drift detection is the construct that resisted implementation most instructively. The framework assumes the specification is scoped per story, which is what makes an amendment attributable. The implementation held the technical specification as one file per project, so an edit could not be attributed, and the flagging mechanism degraded into a cascade that flagged everything downstream on any change. Removing it was the right call, but the finding is not that the feature was hard: it is that a framework construct silently depends on a granularity assumption the framework never states explicitly, and the instantiation is what exposed the omission. That is formative evaluation feeding back into the method, and it is recorded in Section 7.4 as such.

7.4 Design History and Practitioner Feedback

Feedback gathered while building Apex changed the framework, not only the tool. Recording that path is what makes the DSRM iteration explicit rather than implied, and it is reported here as observation, change and rationale rather than as a list of releases. Changes that were made and later reverted are included, because a reverted change is evidence about the design rather than an embarrassment to be omitted.

Two changes were framework-level and are the clearest evidence that evaluation fed back into the method. The first was the **removal of a role**. An earlier version of the framework named a DevOps Alliance, an external group holding deployment approval and the security review of generated infrastructure code. Feedback observed that the framework was intended for anyone with a development background, working alone or in a small team, and that a role defined only by what it supervises can be assumed by the process and never staffed by the organisation. The role was removed and its two responsibilities reassigned to whoever performs the deployment, self-checked where the team is one person. The rationale is argued in Section 6.5; what matters here is that the change originated in observation of use rather than in analysis.

The second was the **demotion of a mandatory step**. Phase 2 originally required a separate sign-off from named design and technical leads before design could be locked. The same observation applied: role-gating a gate presumes a staffed organisation, and on a solo project it either blocks the work or is satisfied by the same person twice, which teaches the practitioner that gates are ceremonial. The step was removed and the gate redefined against recorded evidence rather than against a second signature, which is the form it takes in Table 6.5. The framework's position that a gate's value lies primarily in forcing an explicit act of validation, and only secondarily in who performs it, is a direct consequence of

this iteration.

Three tool-level changes are worth recording for the same reason. A **design-conflict detector** was built, then found to flag conflicts that were not conflicts, since two stories in the same epic legitimately share files; it was narrowed to cross-epic file overlap and endpoint collision, and later removed entirely. **Spec-drift flagging** was built, cascaded as described in Section 7.3, and was removed while its underlying audit trail was kept. An **authentication flow** for one design integration was implemented and then deliberately reverted, because it required an operator to register an application and hold a client secret in backend configuration, which is incompatible with the zero-configuration setup the tool otherwise offers; the simpler token-based path was restored as the final decision rather than left as an unfinished half-feature.

The nature of that feedback should be stated plainly, because it bears on how much weight the design history can carry. It was informal and unstructured: conversations with practitioners during construction, held without an interview guide, without a fixed set of questions, and without recording. Participants are not identified, not because identities were removed afterwards but because they were never collected. There were no numbered rounds and no per-session record, so no table of dates, participant profiles and attributable changes is offered here; constructing one after the fact would impose a structure on the process that the process did not have, which would misrepresent it more than the absence does.

What follows from that is a real limit and it belongs here rather than in Chapter 9. The changes below are traceable in the codebase and in the framework document, so what changed is verifiable; who prompted each change, how many people raised it, and whether a given observation was widely held or came from one person are not recoverable. The design history is therefore evidence that the method was revised in response to use, which is the DSRM claim it is offered for, and it is not evidence about the distribution or strength of practitioner opinion. That second question is what the structured interviews in Chapter 9 exist to answer, and they are a separate instrument with a written guide, a defined participant profile and their own analysis method; the two should not be read as the same evidence gathered twice.

7.5 Quality Assurance of the Implementation

Testing is layered by what each layer can actually catch. At the time of writing the backend carries roughly 1,591 tests, the frontend roughly 304 unit tests, and there are 17 end-to-end specifications; these counts are a snapshot of a codebase that continues to change and are reported as such rather than as a property of the system. The backend suite covers workflow logic, security boundaries and state machines, and depends on a set of automatically applied fixtures that remove categories of flaky-test risk by construction: coordination is forced off so no test depends on network reachability, project-management authentication is bypassed so route tests exercise business logic rather than third-party authentication, storage is pinned to local disk so no test can reach cloud storage, and the two context variables carrying per-request project isolation are reset between tests using the same mechanism production relies on. Frontend end-to-end specifications intercept at the browser network layer rather than by substituting modules in a test runner, because the requests being intercepted are the ones a real browser issues during real navigation.

Five independent jobs must pass in CI before any container image is built, and the deployment job captures the currently live image tags before changing anything, so that an automatic rollback target

always exists. One of the five is a real-stack smoke test that boots the actual processes and asserts on live behaviour rather than on mocks, and it is deliberately narrow: it checks that an invalid token is rejected both with and without the header that selects a project management instance, because a prior regression allowed an invalid token to reach workspace data when validation anchored on the wrong value.

The defects worth reporting here are the ones with methodological significance rather than the ones that were merely difficult. Three concern the framework's own premise that generated output requires validation. A coverage badge reported the model's own claim about which scenarios it had covered, at face value; it is now reconciled server-side against the parsed specification, and a claimed scenario that does not match a real one is dropped rather than displayed. A section-parsing routine anchored on the writer's own headings, which the model also emits, and truncated a live project's technical specification part-way through, silently shortening the contract every later phase reads. And an automation step generated a test plan and then marked the story's quality gate as passed without any verification behind it, writing a fabricated result into the story index; fixing it exposed a second defect immediately downstream, where the deployment step assumed every story reaching it had genuinely passed.

Two others concern isolation, and they matter because isolation is what makes the specification store trustworthy. The identity anchor determining which tenant a request belongs to had to be corrected twice, once because a per-request value was allowed to override a validated one and once because a user-writable stored value was allowed into the chain at all. Separately, a listing endpoint belonging to an integrated tool was found to return pages from projects other than the one requested, because that tool's own filtering does not strictly scope the result set; the fix re-filters every returned row rather than trusting the upstream response. The general lesson is the one the framework states in Section 6.6.2: a control is a measure whose absence makes an incorrect outcome undetectable, and each of these was detected only because something was checked against a reference rather than accepted as reported.

7.6 Limitations of the Implementation

The limitations of Apex as a system must be separated from the limitations of the framework as a methodology, because conflating them would let a defect in one be read as a refutation of the other.

Three are limitations of the system. The original single-writer constraint was a real ceiling rather than a theoretical one, and although the coordination layer described in Section 7.2 has lifted it in production, the fallback path remains the one most deployments would run. The end-to-end test fixtures are not statically checked against the real response contracts, so a new required field can be added to a response with the type checker and the unit suite both staying green; this was not hypothetical, and it is an open gap rather than a solved one. And Apex observes only what passes through it, which is why it can see a story's specification chain but not a defect in production.

Two are limitations that belong to the framework and were exposed by trying to build it. **AI Defect Escape Rate is not instantiable by this class of system**, since it presumes an observation point the framework never says who owns. **Spec-drift detection presumes a granularity the framework does not state**, namely that the technical specification is scoped per story rather than per project; the instantiation revealed the omission by failing on it. Both are reported in Table 7.1 and both are carried into Chapter 9 as findings rather than as defects to be excused.

Where a metric was implemented as a proxy rather than as the definition, the substitution is named: Bolt Cycle Time was computed from story-level gate transitions before it was computed from task-level pack-to-done elapsed time, and for the period in which it was, the figure shown under that name measured queueing and review rather than execution.

8

Demonstration

Contents

This chapter reports the Demonstration activity of the DSRM: the application of the framework, through its reference implementation, to a real software development project. The purpose of the demonstration is to show that the artefact can be used to address the research problem in a genuine setting, and to generate the evidence on which the evaluation in Chapter 9 rests.

8.1 Demonstration Design

This section is written before the demonstration is performed. Committing in advance to what will be observed, and against which criteria, is what separates a demonstration from an anecdote, and it is the only protection against the reasonable suspicion that the observations reported later were selected because they suited the artefact. Where a statement below turns out to be wrong once the work is done, it is corrected in Section 8.4 and the correction is marked, rather than the commitment being quietly revised.

Three settings, and what each can and cannot show

The framework is demonstrated in three settings rather than one, because they fail in different directions and no single one of them carries the claim alone.

The first is **Apex itself**, and the sense in which it is a demonstration needs stating precisely, because it is easy to overstate. The claim is not that Apex's own development followed the framework's phases; it is that Apex is the framework rendered as executable software, so its construction demonstrates that the constructs are realisable rather than merely describable. In DSRM terms the instantiation is itself a solved instance of the problem: the question the framework answers is how AI participation across the lifecycle can be bounded, validated and traced, and a working system in which every phase has a gate, every artefact resolves to a requirement, and no phase can be exited by generation alone is that question answered in a form that can be inspected rather than argued.

What that establishes is real but bounded, and the bound is that it is an existence proof rather than a trial. It shows the constructs can exist; it says nothing about whether they help anyone, whether a team would tolerate them, or whether they survive contact with work the designer did not anticipate. It is also reported retrospectively, since the system already exists, so it is explicitly not pre-registered and nothing in the record below applies to it. Its detail is not repeated here either: which constructs were realised, which were proxied and which resisted implementation is the substance of Section 7.3, and this chapter treats that mapping as the demonstration's first result rather than restating it.

The second is **Outfolio**, a portfolio and project-documentation platform for developers, separate from Apex and from this dissertation. It is the author's own project, so the researcher-role threat below applies to it in full, but it differs from the first setting in the way that matters: the framework is fixed before the work begins, and the software being built has nothing to do with software process, so the framework is applied to a domain it was not derived from. This is the setting in which the full lifecycle, Phases 1 to 6, is exercised end to end.

The third is the **partner organisation** already using Apex, described in Section 8.2. This is the only setting in which the framework is operated by people who did not design it, which is what makes it the strongest of the three and the only one capable of showing that the method is transferable rather than merely usable by its author. Its limitation is coverage: a working team engages the framework where it is useful to them, not exhaustively, so fewer phases will be exercised and the observations will be less complete. Following the consent commitment recorded in Section 9.2, the organisation is not named and identifying project detail is generalised.

Unit of analysis, scope and duration

The unit of analysis is **one Unit of Work carried from Strategic Alignment through to a deployed increment**, not a whole project and not an individual task. That level is chosen because it is the smallest unit at which every gate in Table 6.5 is exercised at least once, and because the framework's own claims are made about the passage of work through phases rather than about the output of any single phase.

Each setting contributes a defined number of such units, fixed here rather than after the fact: Outfolio contributes at least two, so that a second unit can be observed after the first has revealed how the framework behaves in that context, and the partner organisation contributes at least one. The instantiation setting has no unit count, since it is an existence proof rather than a passage of work and is

reported through the construct mapping instead. The demonstration runs until those units reach either a deployed increment or an abandonment recorded with its reason, whichever comes first; an abandoned unit is reported, since a framework that cannot carry work to completion in a real setting is a finding.

What is recorded, decided in advance

At every gate crossed, in every setting, the following is recorded: the artefact presented as evidence, the decision taken, the hat that took it, and, where the gate failed, the named phase the work was routed back to and what changed before it returned. Loop-backs are recorded as first-class events rather than as exceptions, for the reason stated in Section 6.4: a demonstration in which nothing was rejected and nothing looped back is evidence that the framework was not exercised, not evidence that it works.

Alongside the gate record, three further classes of observation are pre-committed. First, the governance metrics the framework defines in Section 6.6.4, to the extent the instantiation computes them, with the two it does not compute reported as unavailable rather than silently omitted. Second, every point at which generated output was modified or rejected by a human, together with what was wrong with it, since the framework's central premise is that this happens and is caught. Third, every point at which the framework was found awkward, over-specified, or was bypassed in practice; these are recorded as they occur rather than reconstructed afterwards, and they are the most valuable input to Chapter 10.

Two commitments about reporting are made here so that they cannot be reconsidered later. Observations unfavourable to the artefact are reported with the same prominence as favourable ones. And the artefacts themselves, generated scenarios, specifications, task breakdowns and test plans, are reproduced rather than described, so that a reader can judge their quality instead of accepting a characterisation of it.

The researcher's role, and the threat it creates

The author designed the framework, built its instantiation, and is the participant in Outfolio. This is the most serious threat to the validity of this chapter and it is named here rather than deferred to Section 9.5.

The threat is not primarily that the author would misreport what happened. It is that a designer applying his own method is the person least likely to notice where it is awkward, because he already knows what it is for, supplies missing rationale from memory without registering that he is doing so, and reads an artefact as adequate against an intention rather than against what is written. The instantiation setting carries a related but distinct problem: because the framework and the system were designed together, a construct that proved inconvenient could be revised in the method rather than recorded as a difficulty, and Section 7.4 reports two occasions on which exactly that happened.

Three things mitigate it and none of them removes it. Pre-registering the record above fixes what counts as an observation before the observations exist, which is what makes a later omission visible. The gate record is defined against artefacts that exist independently of the author's account of them, so a claim in this chapter can be checked against the specification store rather than taken on trust. And the third setting is operated by people who did not design the framework, which is the only structural, rather than merely procedural, mitigation available; where the first two settings and the third disagree, the third is given more weight, and any such disagreement is reported.

What remains unmitigated is stated plainly: for the instantiation and for Outfolio, this chapter reports what a designer observed while using his own design, and no arrangement within this study makes that equivalent to independent application.

8.2 Context

Describe the organisational setting, the team composition and the hats each participant wears, the existing tooling and process baseline, and the nature of the software being built. Establish the baseline concretely enough that a reader can judge what the framework changed.

Characterise the problems present in the setting before the framework was applied, mapping them back to the gaps identified in Chapter 4: inconsistent validation of generated code, upstream pressure on requirements as prototyping accelerates, testing complexity, and unclear responsibility boundaries over generated artefacts.

8.3 Application of the Framework

Walk each phase in turn. For each: what the AI produced, what the human accepted, modified or rejected, which gate was applied, and what the gate decided. Include real artefacts - generated scenarios, specifications, task breakdowns, test plans - as figures or listings rather than describing them.

Record the loop-backs explicitly. A demonstration in which nothing was rejected and nothing looped back is not evidence that the framework works; it is evidence that it was not exercised.

8.4 Observations

Report what was observed at the decision points defined in Section 8.1, including the governance metrics the framework defines. Report observations that were unfavourable to the artefact with the same prominence as those that were favourable.

Where the framework proved awkward, over-specified, or was silently bypassed in practice, record it. These observations are the most valuable input to the refinement discussion in Chapter 10.

8.5 Summary

Summarise what the demonstration establishes and, equally importantly, what it does not. Hand over to Chapter 9.

9

Evaluation

Contents

This chapter reports the Evaluation activity of the DSRM. Following Pries-Heje et al.'s distinction between when and how an artefact is evaluated [25], the evaluation carried out here is ex post, since it assesses the artefact after construction and on the basis of its actual use, and naturalistic, since it takes place in a real organisational setting with real users rather than in a laboratory.

9.1 Evaluation Strategy

Two artefacts were produced in Chapters 6 and 7: a framework and its instantiation. They are not evaluated by the same means, and conflating them would weaken both claims. Table 9.1 therefore states which instrument evaluates which artefact, and which research question it addresses.

State plainly that SUS and NASA-TLX measure the instantiation, not the methodology, and that the framework's own validity rests on the interviews, the analytical assessment, and the demonstration evidence. Positioning this limitation as a deliberate design of the evaluation is far stronger than conceding it under questioning.

Table 9.1: Evaluation instruments, target artefact, and research question addressed

Instrument	Artefact evaluated	DRQ	What it establishes
Demonstration in a real project	Framework and instantiation	DRQ3, DRQ4	That the artefact is applicable and produces the intended artefacts and decisions
System Usability Scale (SUS)	Instantiation (Apex)	DRQ4	Perceived usability of the supporting tool
NASA Task Load Index (NASA-TLX)	Instantiation (Apex)	DRQ4	Perceived cognitive workload imposed on the practitioner
Practitioner interviews	Framework	DRQ1, DRQ2	Practicality, feasibility, and perceived value in real workflows
Agile and Scrum expert interviews	Framework	DRQ1, DRQ2	Alignment with, and departure from, established delivery methodology
Analytical assessment against criteria	Framework	DRQ1, DRQ2	Degree to which stated design criteria are satisfied

9.2 Participants

Report the number of participants, their roles, seniority, sector, and prior exposure to AI-assisted development. Note that SUS is generally considered to yield stable results from around twelve respondents, and state honestly what the achieved sample supports and what it does not.

Describe how participants were recruited: the channels used, how many were approached, and how many of those completed a session. State whether the sample was drawn from a single organisation or topped up from another population, since that determines what the external-validity claim can be.

No submission to an institutional ethics committee was required. The supervisor confirmed that for a study of this kind informed consent, together with anonymisation and a stated retention period, is sufficient at Instituto Superior Tecnico (IST), and that no standard institutional consent form exists that would supersede the one used here. That form is reproduced in Appendix B.

Every participant gave informed consent before any data was collected. The form states what the session involves, what is collected, that the artefact rather than the participant is under test, and that participation may be stopped at any point without giving a reason and without consequence. Consent is recorded as three separate items rather than one, so that agreeing to take part does not imply agreeing to be quoted or to be contacted for a follow-up interview. The three consents were given as tick boxes on the first page of an online form, completed before the task script was opened, so that no response was collected ahead of consent.

No session was observed and nothing was recorded. There is no screen capture, no audio, and no video, either of the task sessions or of the interviews, and the consent form contains no recording item because there was nothing to seek permission for. What was collected is the questionnaire responses,

the timestamps at which each form was submitted, and, for the interviews, contemporaneous written notes taken by the researcher and written up in full immediately after each interview. The methodological cost of this choice, principally that a set of written notes is selective in a way a full transcript is not, is set out in Section 9.5 rather than left for the reader to infer.

Participants are identified only by a code of the form P1 to Pn. Names, employers, and identifying project details were not recorded with the responses, and quotations reproduced in this chapter are anonymised, with any detail that could identify a participant or their organisation removed or generalised. Only wording written down verbatim at the time is presented as a quotation; the remainder of the interview material is presented as paraphrase and is marked as such. The anonymised questionnaire responses and interview notes are retained, accessible only to the researcher, and are reproduced in Appendix B, so that the aggregate figures reported in Section 9.4 can be independently recomputed.

9.3 Instruments and Procedure

The usability study was administered entirely unmoderated. Each participant worked alone, at their own pace, from a fixed written task script, and submitted the questionnaires as online forms without the researcher present. A second, moderated arm of a small number of observed think aloud sessions was designed and deliberately dropped. Two reasons decided it. First, no session was to be recorded, and an observed session without a recording yields notes written by a facilitator who is simultaneously facilitating, which is the weakest form of the only evidence such an arm exists to produce. Second, and more seriously, the author designed both artefacts under evaluation; an unmoderated design removes that conflict structurally rather than mitigating it, because there is no facilitator present to steer, hint, or react. What the decision costs, namely measured task completion, time on task, assist counts, and any observational account of where participants struggled, is stated in Section 9.5. No result reported in this chapter derives from observation.

9.3.1 System Usability Scale

The SUS [52] is a ten-item instrument scored on a five-point scale of strength of agreement, administered after the participant has completed a defined set of tasks. Participants responding in Portuguese are given the validated European Portuguese version [53], with the only adaptation being the substitution of the system name for the generic referent used in the published items; participants responding in English are given the original wording. The two language versions are reported separately rather than pooled.

Report the scoring procedure and state the interpretation scale used, including the adjective ratings of Bangor et al. [54] and the curved grading scale of Sauro and Lewis [55], so that a single number is not left to speak for itself. Report the Usability and Learnability sub-scales separately [56]. State plainly that the score is not a percentage. State also the limits the Portuguese validation itself reports: construct validity was established, but inter-rater reliability was weak (ICC = 0.36) and agreement was 76.67 per cent, and the validation sample was drawn from the general community rather than from software practitioners.

9.3.2 NASA Task Load Index

The NASA-TLX [57] measures perceived workload across six subscales: mental demand, physical demand, temporal demand, performance, effort and frustration. This evaluation uses the unweighted Raw NASA-TLX variant [58], omitting the fifteen pairwise weighting comparisons. The instrument is administered once after each of the marked tasks rather than once at the end of the session, which is the instrument's intended per-task use and yields a workload profile per lifecycle phase rather than a single aggregate.

The Raw variant follows from that choice of administration. The weighting procedure requires fifteen pairwise comparisons, which under per-task administration would mean ninety comparisons per participant, well beyond what the session can absorb. The weights also earn their cost primarily when workload is compared across experimental conditions, and this evaluation has no second condition against which they would do that work.

The variant is not, however, adopted only on grounds of feasibility. Reviewing five hundred and fifty studies twenty years after the instrument's publication, Hart records that removing the weighting is the most common modification made to the NASA-TLX, and that across the twenty-nine studies in which the two versions were directly compared the Raw variant was found to be more sensitive, less sensitive, or equally sensitive depending on the study, with no consistent advantage to weighting [58]. The subscale scores, which are this evaluation's primary result, are in any case identical under both versions, since the weighting affects only how they are combined into a single aggregate.

Two departures from the published instrument are recorded here rather than left to be noticed. Workload was collected on a linear scale from 0 to 10 rather than on the original twenty-interval line, and the responses were rescaled to the instrument's 0 to 100 range by multiplying by ten; no form tool suitable for an unmoderated study reproduces a twenty-interval line. The subscales were also presented in the order mental, physical and temporal demand, effort, frustration and performance, placing the reversed performance scale last rather than fourth, because a reversed scale encountered in the middle of five forward ones, with no facilitator present to explain it, is the most plausible way a participant inverts a sixth of the data set. Neither change affects the unweighted Raw NASA-TLX computation. The first, however, reduces the resolution of the instrument: every response is a multiple of ten, so the resulting values are not directly comparable to studies using the original response scale, and small differences between cells of the workload matrix are not interpreted as meaningful.

Repeated administration carries its own cost, and it is a genuine trade rather than an oversight. Rating the same six dimensions six times invites anchoring, where later ratings drift towards earlier ones, and fatigue on the final tasks of the session. Neither is avoidable under per-task administration; both are carried into Section 9.5 and weighed there against the workload profile that per-task administration is what makes obtainable.

Define the task set both instruments are administered against, and keep it identical across participants. Place the full instruments in an appendix and reference them here.

9.3.3 Semi-Structured Interviews

Interviews were conducted as a session separate from the task work, so that neither fatigue from an hour of tasks nor the immediate impression of the tool dominated the answers. They were not recorded.

The record is contemporaneous written notes, expanded into a full write up on the same day, from which the themes were derived; the codebook was developed from the first two write ups in each participant group and then applied across the remainder, with codes added as they emerged. The interview guides are reproduced in Appendix B.

Report the two participant groups (practitioners and Agile experts), how many were interviewed in each, and the duration actually achieved. Then report the themes with representative anonymised material, distinguishing verbatim quotation from paraphrase, and giving disconfirming evidence the same prominence as supporting evidence.

9.3.4 Analytical Assessment

State the success criteria in advance - clarity, completeness, alignment with SDLC phases, support for human-in-the-loop practice, and governance of AI-enabled activity - and the three-level ordinal scale used to assess each. Each rating must carry a short qualitative justification grounded in observed usage, artefacts, or interview evidence rather than in the author's judgement alone.

9.4 Results

Report SUS results: per-participant scores, mean, and dispersion. Given the expected sample size, report the distribution rather than the mean alone, and avoid inferential statistics the sample cannot support.

Report NASA-TLX results per subscale, not only the aggregate. The subscale profile is the informative part: a tool that raises mental demand while lowering frustration and effort tells a specific and reportable story about where the framework moves the practitioner's burden.

Report the interview findings as themes, with representative anonymised quotations. Report disconfirming evidence explicitly.

Report the analytical assessment as a table of criteria, ratings, and justifications.

9.5 Threats to Validity

Address construct, internal, external, and conclusion validity. Name specifically: the author's dual role as designer and participant, and that the unmoderated design removes it during the sessions but not from the task set, the questionnaire wording, or the interviews, all of which the author authored and conducted; the small and non-random sample, recruited through the supervisor's professional connections, which makes it a convenience sample and bounds what the external-validity claim can be; the single organisational setting; the short observation window, which precludes any claim about long-term maintainability or organisational change; and the fact that the instruments measure perception rather than delivered software quality. For each threat, state the mitigation applied, or state plainly that none was possible.

Address the two threats that follow from the decision to observe and record nothing, and state them as consequences of a deliberate design rather than as oversights. First, there is no observational evidence at all: no measured completion rate, no time on task beyond an indicative form timestamp that cannot distinguish a hard task from an interruption, no assist or error counts, and no independent check on whether a participant understood the deployment refusal in task eight. This leaves standing, rather than partly answering, the threat that the instruments measure perception rather than delivered software quality. Second, the interview analysis rests on written notes rather than transcripts, and note taking is selective in a way transcription is not, with the selection made by the person who designed the artefact.

9.6 Discussion

Synthesise across instruments. Where they agree, say so; where they disagree - for instance a favourable usability score alongside interview scepticism about the process overhead - treat the disagreement as a finding rather than smoothing it over, and read it against the threats to validity stated above.

9.6.1 Answering the Research Questions

Answer DRQ1, DRQ2, DRQ3 and DRQ4 from Chapter 1 in turn, each grounded explicitly in the evidence reported above (results, demonstration, interviews) rather than merely asserted. DRQ1 and DRQ2 concern the framework and are answered primarily from the practitioner and Agile-expert interviews and the analytical assessment; DRQ3 is answered from the construct-to-mechanism mapping in Section 7.3 read together with the implementation limitations, reporting which constructs were realised in full, which were realised as proxies, and which resisted implementation, since the existence of Apex alone establishes only that something was built; DRQ4 is answered from SUS, NASA-TLX, and the demonstration. This closes the loop the introduction opened: each research question is answered here with evidence, not merely restated in the conclusion.

10

Conclusion

Contents

10.1 Summary and Contributions

Restate the problem, the artefacts produced, and what the evaluation established. Summarise, rather than re-derive, how each research question from Chapter 1 was answered; the evidence-grounded answers themselves belong in Section 9.6.1. Keep this to what the evidence actually supports.

10.2 Communication

This section addresses the Communication activity of the DSRM.

Report the dissemination plan and its current status: the scientific article reporting the SLR and its target venue; the report presenting the framework; and this dissertation, formally presented and defended before an academic examination committee. Update the status of each submission before final delivery.

10.3 Limitations

State the limitations of the research honestly: the framework's core constructs are an original synthesis rather than being drawn wholesale from a single validated source; several grounding sources are recent, vendor-reported, or not yet peer-reviewed, and should be cited as such; the governance-standards vocabulary is used as shared language and not as a compliance claim; and the evaluation covers early adoption in a single setting, so it cannot speak to sustained organisational change.

10.4 Future Work

Derive future work from the evaluation findings rather than listing generic extensions. Candidates: application across multiple organisations and team sizes to test generalisation; longitudinal measurement of defect escape and maintainability, which the current evaluation window cannot reach; support for project management backends beyond the one currently integrated; and replacing proxy metrics with direct measurement where production telemetry becomes available.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the author used Claude Code, with the Sonnet 5, Opus 5 and Fable 5 models, together with Google Gemini and Google NotebookLM, to improve the wording and clarity of the manuscript and to support the organisation and review of source material. After using these tools, the author reviewed and edited the text as needed and takes full responsibility for the content of the publication. The final manuscript reflects the author's own analysis, results, and conclusions.

This declaration concerns the use of these technologies in the writing process only. The use of generative AI as the object of study of this research, and within the artefacts it produces, is described in Chapter 6 and Chapter 7.

Bibliography

[1] J. H. Moore and N. Tatonetti, “Vibe coding: a new paradigm for biomedical software development,” *BioData Mining*, vol. 18, no. 1, pp. 1–3, 2025.

[2] A. Zamfiroiu, C.-B. Păştinică, C. Crăciun, C. Ciutacu, and T. Luca, “Exploring Conversational Programming through the CHOP Paradigm and Artificial Intelligence,” *Informatica Economica*, vol. 29, no. 2, pp. 5–17, 2025.

[3] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia, “Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–27, 2025.

[4] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang, “Empowering Agile-Based Generative Software Development through Human-AI Teamwork,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 6, pp. 1–46, 2025.

[5] W. Hou and Z. Ji, “Comparing Large Language Models and Human Programmers for Generating Programming Code,” *Advanced Science*, vol. 12, no. 8, pp. 1–12, 2025.

[6] Stack Overflow, “2025 Developer Survey: AI,” Annual practitioner survey, industry-reported, not peer-reviewed, 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai/

[7] DORA and Google Cloud, “State of AI-assisted Software Development 2025,” Industry research report, not peer-reviewed, 2025. [Online]. Available: https://dora.dev/dora-report-2025/

[8] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, “Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks,” arXiv:2512.03262, preprint, not yet peer-reviewed, 2025. [Online]. Available: https://arxiv.org/abs/2512.03262

[9] Veracode, “2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding,” Industry report, not peer-reviewed. Assessment of over 100 large language models across Java, Python, C# and JavaScript, July 2025. [Online]. Available: https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf

[10] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, preprint, not published in a peer-reviewed venue, 2023. [Online]. Available: https://arxiv.org/abs/2302.06590

[11] J. Becker, N. Rush, E. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” arXiv:2507.09089, METR, preprint, not yet peer-reviewed, 2025. [Online]. Available: https://arxiv.org/abs/2507.09089

[12] Faros AI, “The AI Productivity Paradox Report 2025,” Industry telemetry report, not peer-reviewed. Sample of over 10,000 developers across 1,255 teams, July 2025. [Online]. Available: https://www.faros.ai/blog/ai-software-engineering

[13] L. Banh, F. Holldack, and G. Strobel, “Copiloting the future: How generative AI transforms Software Engineering,” *Information and Software Technology*, vol. 183, 2025.

[14] M. Farhanna, N. Paramitha, M. Susi, H. Surya, A. Yanti, P. Dea, and S. Deni, “Exploring the Use of Generative AI in Software Development: A Preliminary Study,” in *E3S Web of Conferences*, vol. 645, 2025, p. 04002.

[15] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, “Research directions for using LLM in software requirement engineering: a systematic review,” *Frontiers in Computer Science*, vol. 7, 2025.

[16] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, “The Current Challenges of Software Engineering in the Era of Large Language Models,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[17] H. A. Al-Hashimi, R. A. Khan, H. S. Alwageed, A. M. Algarni, S. Ayouni, and A. O. Almagrabi, “Exploring the role of generative AI in enhancing cybersecurity in software development life cycle,” *Array*, vol. 28, p. 100509, 2025.

[18] X. Bai, S. Huang, C. Wei, and R. Wang, “Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability,” *Expert Systems With Applications*, vol. 269, 2025.

[19] M. Basha and G. Rodríguez-Pérez, “Trust, transparency, and adoption in generative AI for software engineering: insights from twitter discourse,” *Information and Software Technology*, 2025.

[20] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, “An Empirical Study on Challenges for LLM Application Developers,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–37, 2025.

[21] Y. Dong, X. Jiang, Z. Jin, and G. Li, “Self-Collaboration Code Generation via ChatGPT,” *ACM Transactions on Software Engineering & Methodology*, vol. 33, no. 7, pp. 1–38, 2024.

[22] U. K. Durrani, M. Akpinar, M. F. Adak, A. T. Kabakus, M. M. Ozturk, and M. Saleh, “A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023),” *IEEE Access*, vol. 12, pp. 171 185–171 204, 2024.

[23] Y. Han and C. Lyu, “Multi-stage guided code generation for Large Language Models,” *Engineering Applications of Artificial Intelligence*, vol. 139, 2025.

[24] M. A. Haque, “LLMs: A game-changer for software engineers?” *BenchCouncil Transactions on Benchmarks, Standards & Evaluations*, vol. 5, no. 1, pp. 30–41, 2025.

[25] J. R. V. Jan Pries-Heje, R. Baskerville, “Strategies for Design Science Research Evaluation,” *European Conference on Information Systems (ECIS)*, no. 1, pp. 1–13, 2008.

[26] S. K. Jabrw and Q. I. Sarhan, “A Systematic Survey on Large Language Models for Code Generation,” *ARO-The Scientific Journal of Koya University*, vol. 13, no. 2, 2025.

[27] V. V. Jensen, A. Alami, A. R. Bruun, and J. S. Persson, “Managing expectations towards AI tools for software development: a multiple-case study,” *Information Systems & e-Business Management*, pp. 1–33, 2025.

[28] U. Karlovs-Karlovskis, “Generative Artificial Intelligence Use in Optimising Software Engineering Process: A Systematic Literature Review,” *Applied Computer Systems*, vol. 29, no. 1, pp. 68–77, 2024.

[29] D.-K. Kim and H. Ming, “Assessing output reliability and similarity of large language models in software development: A comparative case study approach,” *Information and Software Technology*, vol. 185, 2025.

[30] D.-K. Kim, “Improving Software Development Traceability with Structured Prompting,” *Journal of Computer Information Systems*, pp. 1–21, 2025.

[31] J. Li, G. Li, Y. Li, and Z. Jin, “Structured Chain-of-Thought Prompting for Code Generation,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 2, pp. 1–23, 2025.

[32] V. S. Pendyala and N. B. Thakur, “Performance and interpretability analysis of code generation large language models,” *Neurocomputing*, vol. 656, 2025.

[33] K. Qiu, N. Puccinelli, M. Ciniselli, and L. Di Grazia, “From Today’s Code to Tomorrow’s Symphony: The AI Transformation of Developer’s Routine by 2030,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

[34] T. Ramos, A. Dean, and D. McGregor, “AI-Augmented Software Engineering: Revolutionizing or Challenging Software Quality and Testing?” *Journal of Software: Evolution & Process*, vol. 37, no. 2, pp. 1–7, 2025.

[35] V. Terragni, A. Vella, P. Roop, and K. Blincoe, “The Future of AI-Driven Software Engineering,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–20, 2025.

[36] A. Tkachuk, “Determining the Capabilities of Generative Artificial Intelligence Tools to Increase the Efficiency of Refactoring Process,” *Technology Audit & Production Reserves*, vol. 3, no. 2(83), pp. 6–11, 2025.

[37] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, “A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

[38] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, “Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–25, 2025.

[39] S. H. Yoo and H. J. Kim, “Security Analysis of Automated Code Generation: Structural Vulnerabilities in AI-Generated Code,” *Tehnički Glasnik*, vol. 19, no. 4, pp. 560–574, 2025.

[40] Thoughtworks, “Spec-driven development,” Technology Radar, Vol. 33, November 2025. [Online]. Available: https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development

[41] S. E. Farrag, “The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development,” arXiv:2605.01160, preprint, not yet peer-reviewed, 2026. [Online]. Available: https://arxiv.org/abs/2605.01160

[42] Amazon Web Services, “AI-Driven Development Lifecycle (AI-DLC),” Vendor methodology announced at AWS DevSphere 2025. Reported productivity figures are vendor-reported and not independently verified, 2025. [Online]. Available: https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/

[43] H. Kniberg and A. Ivarsson, “Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds,” Whitepaper, industry-reported, not peer-reviewed. A description of Spotify’s organisation at the time of writing, not a prescriptive method, November 2012. [Online]. Available: https://blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify

[44] Leading Complexity, “Summary of Henrik Kniberg’s session, about his thoughts on the Spotify Model, in the Leading Complexity programme 2022,” Crisp’s Blog. Third-party published summary of Henrik Kniberg’s session in the Leading Complexity programme 2022, not written by Kniberg himself, February 2023. [Online]. Available: https://blog.crisp.se/2023/02/06/leadingcomplexity/leading-complexity-series-summary-1-henrik-knibergs-thoughts-on-the-spotify-model

[45] J. Lee, “Failed #SquadGoals,” Practitioner account by a former Spotify employee, industry-reported, not peer-reviewed, April 2020. [Online]. Available: https://www.jeremiahlee.com/posts/failed-squad-goals/

[46] M. Skelton and M. Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. Portland, OR: IT Revolution Press, 2019.

[47] C. Bryar and B. Carr, *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. New York, NY: St. Martin’s Press, 2021.

[48] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” National Institute of Standards and Technology, U.S. Department of Commerce, Technical Report NIST AI 100-1, January 2023. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[49] ISO/IEC, “Information technology - Artificial intelligence - Management system,” International Organization for Standardization/International Electrotechnical Commission, International Standard ISO/IEC 42001:2023, December 2023.

[50] S. Siddeeq, M. Abbasi, J. Rasku, Z. Zhang, F. Christophe, T. Mikkonen, and P. Abrahamsson, "Epic-Organized vs. Requirement-Aligned Gherkin: An Empirical Evaluation of LLM-Based Acceptance Criteria Generation," Accepted at the International Conference on Software Engineering of Emerging Technologies (SEET 2026), Springer Nature. arXiv:2607.01980, 2026. [Online]. Available: https://arxiv.org/abs/2607.01980

[51] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of Developer Productivity: There's More to it than You Think," *ACM Queue*, vol. 19, no. 1, pp. 20–48, 2021.

[52] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. Taylor and Francis, 1996, pp. 189–194.

[53] A. I. Martins, A. F. Rosa, A. Queirós, A. Silva, and N. P. Rocha, "European Portuguese validation of the System Usability Scale (SUS)," *Procedia Computer Science*, vol. 67, pp. 293–300, 2015. [Online]. Available: https://doi.org/10.1016/j.procs.2015.09.273

[54] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

[55] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Morgan Kaufmann, 2016.

[56] J. R. Lewis and J. Sauro, "The factor structure of the System Usability Scale," in *Human Centered Design, HCII 2009, Lecture Notes in Computer Science*, vol. 5619, 2009, pp. 94–103.

[57] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. North-Holland, 1988, pp. 139–183.

[58] S. G. Hart, "NASA-Task Load Index (NASA-TLX); 20 years later," in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, 2006, pp. 904–908.

A

Extended Results for the Systematic Literature Review

This appendix holds the evidence tables underlying the review reported in Chapter 4. They are reproduced here rather than in the chapter itself so that the chapter can state what the review found without the reader having to read four tabulations to reach it, while the tabulations remain available in full for anyone auditing the synthesis. Kitchenham's guidelines require the review to be reconstructible from the protocol together with this material.

A.1 Distribution of the retained corpus by venue

Table A.1: Journals and Publication Counts

Journals	Nr. Publications
ACM Transactions on Software Engineering & Methodology	10
Information and Software Technology	3
Advanced Science	1
Applied Computer Systems	1
ARO-The Scientific Journal of Koya University	1
Array	1
BenchCouncil Transactions on Benchmarks, Standards & Evaluations	1

Continued on next page

Journals	Nr. Publications
BioData Mining	1
E3S Web of Conferences	1
Engineering Applications of Artificial Intelligence	1
Expert Systems With Applications	1
Frontiers in Computer Science	1
IEEE Access	1
Informatica Economica	1
Information Systems & e-Business Management	1
Journal of Computer Information Systems	1
Journal of Software: Evolution & Process	1
Neurocomputing	1
Technology Audit & Production Reserves	1
Tehnički Glasnik	1

A.2 Stakeholder opinions reported in the literature

The fourth research question is answered in Chapter 4 from the three tables below, which separate the perspectives by the group holding them: the public and researchers, developers and practitioners, and organisations and management. The separation is retained because the three groups disagree, and the disagreement is itself a finding.

Table A.2: Public / Researchers Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
General Trust / Perception	People's perception of systems powered by artificial intelligence as a mixture of "software and sorcery." [3]	6	[2, 3, 18, 19, 32, 34]
Misinformation Risk	"Users highlight its potential to misguide or hallucinate to students." [19]	4	[13, 19, 22, 26]
Bias and Impartiality Concerns	Users express dissatisfaction over perceived "leftist bias and propaganda in ChatGPT." [19]	3	[3, 19, 34]
Output Diversity / Similarity	"57% similarity overall with 43% variability, presenting that each model has its unique characteristics." [29]	2	[4, 29]
Legal, Copyright, and Licensing Issues	"Questions also surround the copyrightability/ownership of generated code [...]" [3]	2	[3, 19]
Evaluation Urgency	Urgent need to "establish an effective, objective, and complete evaluation system" for code-generation LLMs. [3]	2	[3, 32]
Job Automation	"Many users fear that AI will eventually replace human developers [...]" [19]	1	[19]

Table A.3: Developers / Practitioners Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Productivity and Efficiency Gains	"The knowledge and reasoning capabilities of LLMs have the potential to result in significant efficiency gains." [13]	6	[2,13,19,23, 24,35]
Reliability Issues / Hallucinations	"LLMs sometimes produce highly convincing content that is factually wrong [...] the 'hallucination phenomenon'." [18]	6	[13, 18, 19, 22,26,32]
Output Variability and Trust Decline	"I get a different code each time [...] I lose a bit of confidence in the work." [13]	5	[13, 19, 32, 34,35]
Quality of Generated Code	"Several quality issues: undefined variables, insecure comments, and code requiring significant revision." [26]	4	[4,26,27,35]
Augmentation of Human Capabilities	"LLMs are not merely a product of overhyped marketing; they represent a profound shift in how software is engineered. They should be seen as powerful tools that augment human capabilities rather than replace them." [24]	3	[19,24,35]
Learning and Skill Improvement	LLMs "[...] directly support learning processes through interaction and conceptual clarification." [2]	3	[2,16,24]
Knowledge Acquisition / Search Replacement	"LLMs present data in a concise and contextually relevant manner." [13]	2	[2,13]
Controllability	Participants praised its "controllability [...] allowing modification and feature adjustment after generation." [4]	2	[4,19]
Risk of Over-reliance and Cognitive Decline	"There is almost the danger that you stop thinking for yourself [...] people will tend to become lazy." [13]	2	[13,19]
Investment of time with prompts	"You spend ages trying to cut the prompt so that it comes out right [...]" [13]	2	[13,19]

Table A.4: Organizations / Management Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Need for Human Oversight	LLM outputs "still necessitate extensive human review before implementation." [29]	7	[4,17,19,29, 30,33,35]
Data Security, Privacy, and Corporate Bans	"Due to concerns regarding data privacy, when considering real-world practice, most software organizations tend to avoid using commercial LLMs and would prefer to adopt open source ones with training or fine-tuning using organization-specific data." [37]	6	[13, 15, 19, 35,37,39]

Continued on next page

Operational Costs and Resource Constraints	Organizations must "consider the current limitations in terms of computational power or pay close attention to energy consumption," leading them to "tend to fine-tune medium-sized models". [37]	3	[13, 15, 37]
Strategic Goal for Developer Satisfaction	"We have to be able to compete on efficiency and job satisfaction [...] It is difficult to find software developers." [27]	2	[24, 27]
Expectation Management Requirement	"SDOs must carefully manage their expectations towards AI tools." [27]	2	[19, 27]
Strategic Competitiveness	The "early adoption of LLMs in software engineering is crucial to stay competitive in this rapidly evolving landscape." [24]	1	[24]

A.3 Industry sectors adopting AI and LLMs

The evidence behind the fifth research question, answered in Chapter 4.

Table A.5: Industry sectors adopting AI and LLMs

Industry Sector	Description	#	References
Software Engineering, Software Development & Consulting Services	Software Development and Consulting is mentioned a lot, since they are the main sectors where AI and LLMs are being utilized in many different ways, such as AIOps, Cloud Management, Requirements Engineering, Software Testing, Quality Assurance, Code Generation and Development, Cybersecurity, Vulnerability Management, Refactoring, Code Optimization, Specialized Hardware, Scientific Computing and Emerging Technologies (like Blockchain/Crypto).	25	[2–4, 13–20, 22–24, 26–29, 32, 33, 35–39]
Finance, Banking, and Regulated Industries (Including Insurance)	Specialized LLMs can understand the "regulatory requirements, security needs, and performance constraints" especially in blockchain, cryptocurrency, smart contracts, and regulated environments. [24, 27, 29]	8	[4, 13, 17, 19, 24, 27, 29, 34]
Transportation and Safety-Critical Domains	The lack of transparency of LLMs is problematic, particularly in "safety-critical applications like healthcare, finance, or aerospace, where understanding the reasoning behind code is essential for ensuring security and correctness". [3]	7	[3, 15, 19, 24, 34, 37, 39]
Healthcare, Medical, and Biomedical Research	AI "accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles in biomedical research and clinical application." [1].	5	[1, 4, 5, 18, 19]

Continued on next page

Industry Sector	Description	#	References
Education and Training	LLMs "directly support learning processes through interaction and conceptual clarification". Students should be taught "how to specify AI, including functional, non- functional, and 'happy and unhappy path scenarios'" and learn "prompt engineering workarounds" and to "research and fact- check AI responses". [34]	5	[2,4,5,15, 34]
Legal Reasoning and Regulatory Compliance	LLMs demonstrate success across multiple fields, such as education, healthcare, and legal reasoning. Specifically, prompt engineering is used to "tailor LLMs for the US legal system, enabling the model to identify relevant cases and predict judicial outcomes with greater precision". [26]	4	[17,18,26, 34]
Digital Ecosystems (E-Commerce and Services)	Case studies involve the development of systems such as a food order and delivery system (FODS) and a smart wallet system (SWS). LLMs are applied in contexts like Online Retailers and the service industry. [4,29]	3	[4,17,29]

List the final set of studies retained by the review: identifier, full reference, venue, year, and which research questions each study contributes to. This is the auditability requirement of Kitchenham's guidelines - a reader must be able to reconstruct the review from this appendix together with the protocol reported in Chapter 4.

Include the full search string and the date on which the search was executed, if these are not already stated in the review protocol.

B

Evaluation Instruments

Reproduce the SUS questionnaire exactly as administered, including the response scale and any wording adapted to name the system under evaluation.

Reproduce the NASA-TLX instrument as administered, including the subscale definitions shown to participants and the response scale actually used. The unweighted Raw variant was administered, so there is no pairwise comparison sheet to reproduce.

Include the semi-structured interview guides for both participant groups (practitioners, and Agile and Scrum experts), and the informed consent form.

Include the per-participant raw scores in anonymised form, so the aggregate figures reported in Section 9.4 can be independently recomputed.